ECE 385

SPRING 2025

FINAL PROJECT REPORT

SPACE INVADERS

Artha Amanda Sidauruk
(3220300399)
Cristina Natalia Lumban Tobing
(3220300398)
TEAM 44. D-231
TA: Qiyang Wu

1. Introduction

This project implements a hardware-based Space Invaders arcade game on the DE2-115 FPGA using System Verilog and C. The game transitions through four states: Start, Play, Win, and Lose. The player controls the spaceship located at the bottom of the VGA screen via a USB keyboard. The core objective is to eliminate 30 descending aliens by firing missiles while preserving limited lives by avoiding enemy fire or preventing any alien from reaching the player's position.

The game logic is implemented in System Verilog, while the NIOS II processor is connected to the keyboard via the EZ-OTG controller. The user input will be handled using a USB keyboard control using C code on the SoC, based on the Lab 8 USB framework. The VGA rendering is achieved via color mapping and timing control modules. The system integrates FSMs, collision detection, score/life tracking, and missile control to enable real-time interactive gameplay.

2. Written Description

The game's functionality emerges from the strong connection and cooperation of multiple hardware modules, each performing a distinct task such as control, display rendering, game state transitions, and player interaction. The game is architected around several synchronized modules that exchange status signals to maintain real-time responsiveness. Elements like the spaceship, aliens, missile, and score are designed to behave semi-independently and interact dynamically through shared data pathways to have synchronization.

2.1 Overview

2.1.1 Purpose of the circuit

The primary goal of this circuit is to create the game Space Invaders using digital logic components on an FPGA. The system integrates player controls, enemy behavior, missile handling, collision logic, and VGA display in order to form the complete playable game. The project demonstrates the use of the hardware description and verification language, SystemVerilog, in constructing elaborate real-time systems by implementing interactive embedded designs.

2.1.2 Features

The following **baseline features** have been implemented in our Space Invaders game:

- **Game Start Display**
  The game begins with the title screen displaying the name of the game, Space Invaders, and the prompt instructs the player to press the spacebar on the keyboard to start the game and transition to the gameplay state.

- **State Transitions**
  The game operates through four states: Start, Play, Win, and Lose. State transitions occur depending on player input and in-game conditions. The game begins in the Start state and transitions to the Play state immediately after the player presses the spacebar. After the Play state, the next state is determined by certain conditions that are continuously monitored by the system: the game transitions to Win state if all the enemies are destroyed; the game transitions to Lose state if either player loses all lives or an enemy reaches the player's row.

- **Player Control and Firing**
  The player controls the spaceship using the keyboard input: "A" to move left, "D" to move right, and the spacebar to fire the missile. The spaceship's movement is limited horizontally; that is, it will not move beyond the left and right edges of the screen. Upon pressing the spacebar, the player's missile is immediately launched and moves vertically upward toward the enemy. The number of the player's missiles is limited to three on the screen at any given time; additional firing attempts will be ignored until at least one of the three current missiles disappears (either collides with the enemy or goes off the screen). This limitation ensures reasonable and balanced gameplay in reaching the win condition and the speeds of movement of the spaceship and descending enemies.

- **Missile Management**
  Both the spaceship and the enemy can fire the missile vertically. The creation, movement, and lifetime of all the missiles are managed in a synchronized manner. The system keeps track of the active missiles in the screen and removes them upon collision or once the missiles exit the screen boundaries. Player missiles are triggered via keyboard input, while enemy missiles are produced autonomously based on a specific logic, as detailed in the following section.

- **VGA Rendering**
  The VGA display renders all the game states in real-time, including **Start** (game title and press space prompt), **Play** (spaceship, enemies, missiles, score, lives, mystery alien), **Win** (“YOU WIN” message), **Lose** (“GAME OVER” message). Rendering is frame-synchronized and changes dynamically based on the game state. Sprites for the player, enemies, mystery alien, score, lives, and text messages are loaded from ROM and decoded into pixel data to display at the specific screen location and time. The given “ECE-385 Helper Tools” is also utilized to convert a custom PNG start screen into a hex format.

- **Lives Display**
  At the beginning of the game, the player has three lives. One life is lost each time the player is hit by an enemy missile. The game transitions to a Lose state once all lives are depleted. However, there's still an opportunity for the player to gain life by successfully hitting the

mystery alien, which generates a downward-moving life icon. If the player is able to catch it, one life is restored, but the total number of lives is a maximum of three.

- **Enemy Animation**
  The enemy formation comprises three rows, each containing ten identical alien sprites and animation behaviour. By toggling between two sprite frames over time, the illusion of movement is created, producing the animation that makes the game more interesting. For example, the middle row enemy has a jellyfish shape that appears to sway from side to side.

- **Enemy Missile Firing (AI)**
  During the Play state, the enemy periodically fires the missile vertically downward toward the player. The firing has internal logic to make the enemy missile produced by the enemy closest to the player's horizontal location. This behavior increases the likelihood of player damage and supports game progression by raising the probability of transitioning to the **Lose** state.

- **Collision Detection**
  The game continuously monitors all the missiles in order to detect the collision between any missile, whether the player, enemies, or the mystery alien. If an enemy missile intersects with the position of the player, then the system detects it and immediately reduces the player's life. Suppose the player's missile hits either enemies or mystery aliens. In that case, the enemies or mystery alien is removed from the screen, and the player's score increments according to the assigned score for each enemy type or mystery alien. This real-time detection system ensures a responsive game outcome through the accurate interaction between the missile and all the elements in the game.

- **Enemy Elimination & Scoring**
  Each enemy that is hit by a player missile will immediately be eliminated and permanently removed from the screen for the lifetime of the game. Each row of enemies is assigned a distinct score value: +3 for the top row, +2 for the middle, and +1 for the bottom. In real-time, the player’s score is updated on the screen immediately after the collision is detected.

- **Player Damage & Life Reduction**
  If the enemy missile hits the player, one life is reduced, and for several seconds, the player becomes temporarily invincible, indicated by a flashing effect. During this period, the player cannot lose additional lives or collect life bonuses, ensuring protection from consecutive hits. This aims to give the player recovery and repositioning time. If there is no remaining life, the game enters the **Lose** state.

- **Limited Player Lives**
  Once the game starts, the player is assigned three live. Lives are reduced each time the player is hit by an enemy missile, and life is incremented if the player can catch the drop life icon when the mystery alien is hit. However, the player is only allowed to have a maximum of three lives during the game. Once all lives are lost, the system triggers a transition to the **Lose** state.

- **Win Condition**
  The game transitions to the Win state when all 30 enemies are eliminated by the player's missiles.

- **Lose Condition**
  The game enters the Lose condition if either all the player's life is depleted or if any enemy reaches the player's row.

The following **additional features** have been implemented in our Space Invaders game:

- **Enemy Types with Different Scores**
  The game has three categories of enemies, each marked by their visual and associated score value. Enemies positioned in the top row are worth 3 points, while the middle and bottom rows award 2 and 1 points, respectively when destroyed. This tiered scoring encourages the player to have strategic targeting to maximize the score value.

- **Downward Moving Enemy and Lose Condition**
  All enemies move downward toward the player every 2 seconds. The game enters the Lose state immediately if any enemy reaches the player's row, irrespective of the remaining lives. Such a mechanic adds time pressure and forces the player to hit the enemy as quickly as possible to prevent loss.

- **Mystery Alien Bonus Scoring**
  A mystery alien appears at the top of the screen as soon as the top-row enemy moves downward by one row. The mystery alien then moves in a horizontal direction quickly through the screen from left to right. The player gets a 10 score if they successfully hit the mystery alien.

- **Mystery Alien Drops Extra Life**
  Destroying the missing alien with a player's missile activates an additional feature that drops a life icon. This icon, which signifies an extra life, falls vertically from the mystery alien's last position. If the player catches the heart, the player gains one life, up to a maximum of three lives. There is also an invincibility of mystery aliens to avoid

consecutive hits producing consecutive extra life. This feature encourages the player to target the mystery alien while managing the time to hit other enemies before touching the spaceship's position and avoid enemy missiles while waiting for the drop of extra life until it reaches the bottom of the screen.

2.2 Data Flow of The Circuit

2.2.1 Inputs and Outputs Summary

This circuitry takes in a set of external inputs and gives outputs related to the visual display and status. The main input consists of:

- **CLOCK_50**
  50 MHz system base clock frequency.
- **KEY[0]**
  Active low reset is implemented using a physical push-button.
- **OTG_INT, OTG_DATA**
  Signals interfacing with the CY7C67200 USB controller.

Primary outputs include:

- **VGA_R, VGA_G, VGA_B**
  8-bit RGB color outputs.
- **VGA_HS, VGA_VS, VGA_CLK, VGA_BLANK_N, VGA_SYNC_N**
  VGA timing and synchronization signals.
- **HEX0, HEX1**
  Seven-segment display outputs to display the current keycode input.

In addition to physical I/Os, the system utilizes internal data paths such as life_count, enemy_active_flags, mystery_exists, enemy_missile_active, and game state to manage game logic and visual outputs.

2.2.2 Input and Output Processing Logic

Signals and inputs are processed in parallel using logic synchronous to the vertical sync signal (VGA_VS), which is the primary clock for the game loop. Each of the multiple hardware modules implements a specific gameplay function, and data intercommunication occurs among the modules.

The keyboard input is fetched on the USB OTG interface. There is a NIOS II soft processor whose firmware is designed to pull the USB keyboard connected through the EZ-OTG chip. The necessary keycodes are placed in a memory-mapped IO register (keycode_export) associated with hardware logic. This allows the transfer of keyboard input to the player module for movement and missile firing.

Enemy behavior is coordinated by the enemy_logic, which computes horizontal and vertical shifts and generates animation offsets for sprite toggling

Missile behavior is split into two modules:

- player_missile_ctrl: creates and tracks missiles fired by the player.
- enemy_missile_ctrl: Control the missiles produced by the enemy, which are periodically fired targeting the player's column.

A collision is handled by the detect_collision module. This module checks the missile's impacts on enemies, the player, and the mystery alien. The collision flag, enem_coll, player_collision, and mystery_hit are set high for one clock cycle to signal events.

The overall game states are controlled by game_fsm, which manages transitions between states: Start, Gameplay, Win, and Lose, based on real-time conditions. For this Purpose, condition such as enemy_flags == 0, life_count == 0, and enemy_y_offset are used to determine the state transitions.

The mystery_controller handles the movement and spawning of the special alien ship. If hit, it triggers the increment of the score by +20 points and the life_drop_controller, which spawns a falling life icon that the player is able to catch in order to regain one life.

Score calculation is accomplished by score_tracker, which offers scoring metrics for the row of enemies or mystery alien hits based on assigned value. It handles the score as Binary-Coded Decimal, or BCD, and provides the score in a visual format through pixel-mapped rendering.

All modules run simultaneously, exchanging real-time data to change the state, control entities, and render the output.

2.2.3 Visual and Logical Output Representation

The Color_mapper module is solely responsible for generating visual output to the VGA monitor. All elements are drawn to the screen based on internal signals. The current pixel coordinates (DrawX, DrawY) and game state input are simultaneously used to determine the sprite at each pixel.

- **Player ship**: The position is specified by the player module and flashes when the invincible status is active.
- **Enemies**: Rendered in three rows using animation frames from Sprite ROMs.
- **Missiles**: Rendered as a small rectangle. Player missile: moving vertical-upward. Enemy missile: moving vertical-downward.
- **Mystery alien**: Rendered as a 32x32 sprite moving horizontally across the top of the screen at a high speed.
- **Falling Life Icon**: The small rectangle that descends after a player missile hits the mystery alien.

- **Score**: Rendered using pixel-based number ROM, updated in real-time.
- **Lives**: Red heart icons are displayed at the top left based on life_count.
- **Text Graphics**: "SPACE INVADERS PUSH START," "YOU WIN," and "GAME OVER" are rendered based on the current game state.
  All the rendering is synchronized with the VGA controller‘s timing, dynamically producing a smooth and stable visual output.

3. Description of Core Modules

3.1 Core Game Logic and Control

1) **top_level_space_invaders**
   - **Purpose**
     The top_level_space_invaders module is the top-level that integrates all the components of the Space Invaders game hardware system implemented on the DE2-115 FPGA. This module manages all subsystems, such as USB keyboard interfacing, VGA rendering, enemy and player controller, missile controller, score and life tracking, and game state management. In order to instantiate the complete game environment, it creates and interconnects all subordinate modules and external interfaces.
   - **Inputs**
     - *CLOCK_50*: 50 MHz system clock input for main timing and synchronization.
     - *[3:0] KEY*: KEY[0] is used to reset the game (aynchronous active-low).
     - *OTG_INT*: Interrupt from CY7C67200 USB controller.
   - **Outputs**
     - *HEX0, HEX1*: 7-segment to display the keycode input.
     - *VGA_R, VGA_G, VGA_B* : signal of VGA color.
     - *VGA_CLK, VGA_SYNC_N, VGA_BLANK_N, VGA_VS, VGA_HS*: synchronization signals VGA for display
     - *OTG_ADDR, OTG_CS_N, OTG_RD_N, OTG_WR_N, OTG_RST_N*:
       Signals are used to control the USB keyboard.
     - *DRAM_ADDR, DRAM_BA, DRAM_DQM, DRAM_RAS_N, DRAM_CAS_N, DRAM_CKE, DRAM_WE_N, DRAM_CS_N, DRAM_CLK*:
       Signals used to access the external SDRAM by NIOS II.
   - **Inputs**
     - *OTG_DATA*: bidirectional data bus for USB keyboard.
     - *DRAM_DQ*: bidirectional data bus for external SDRAM.
   - **Operation**
     The top_level_space_invaders module is a central control for synchronizing all subsystems. It manages system initialization, game states, inputs, and real-time

synchronization. Upon reset or power up, it sets all corresponding signals and commences by displaying the start screen, waiting for user interaction. The NIOS II processor captures user control inputs using the USB keyboard and sends them to the logic system for processing player control and game state updates.

This module continuously alters the internal game state according to the player's inputs and certain in-game events. It manages the most crucial parts of gameplay, including the player's movement, firing missiles, enemy positioning, and score calculation, based on the system clock and frame synchronized vertical sync pulses from the VGA controller. Collision flags and other conditions, like the number of lives of the players and enemies and their respective positions, determine the four game states: Start, Play, Win, and Lose.

VGA signal generation for visual output is done in a modular manner. The controlling module chooses color outputs according to the active game state and its rendering priority. It also controls auxiliary outputs like the HEX display that serve debugging purposes or key displays. All modules operate concurrently under this framework, with the top-level logic ensuring seamless communication, frame-by-frame synchronization, and deterministic behavior throughout the game lifecycle.

2) game_fsm

- **Purpose**

  The game_fsm module aims to control the game state transitions: Start, Play, Win, and Lose, depending on the input and in-game conditions.

- **Inputs**
  - *frame_tick*: vertical sync signal for the clock.
  - *[29:0] enemy_flags*: encoded the status of 30 enemies.
  - *[2:0] life_count*: store the current number of player's life.
  - *[7:0] kbd_code*: USB keyboard's keycode.
  - rst_game: asynchronous system reset.
  - *[9:0] enemy_y_offset*: vertical enemy's pixel
  - *[5:0] remaining_enemies*: counter for storing a number of enemies.

- **Outputs**
  - *[3:0] curr_state*: current state

- **Operation**

  At every rising edge of the frame_tick signal or during an asserted reset, the state machine checks the game conditions to update the global game state. If *rst_game* is high, the curr_state is initialized to 0 (Start). In state 0, the transition to state 1 (Play) occurs only if the spacebar key, kbd_code == 8'd44, is pressed. In Play state (state 1), the module scans for three game-ending conditions in priority:
  - if remaining_enemies == 0, the game enters the state 2 (Win);
  - if life_count == 0, the game enters the state 3 (Game Over); and

- if the vertical position of the lowest enemy row (enemy_y_offset + 96) is equal to or greater than 416 pixels, the game transitions to Lose.

If none of the conditions are satisfied, the state is held at its current value. Thus, condition-based state management is ensured throughout the game.

3) player_control

- **Purpose**
  The player module controls all the player's behavior: the spaceship's horizontal position, the player's missile firing, the player's life, the invincibility phase of the player, and the response of the player to the life bonus produced by the mystery alien.
- **Inputs**
  - *rst_game*: asynchronous system reset.
  - *frame_tick*: vertical sync signal for the clock.
  - [*7:0] kbd_input*: USB keyboard keycode.
  - *player_collision*: signal is set high for one full cycle when a player is hit.
  - *collected*: signal is set high for one full cycle when the player catches the drop life from the mystery alien.
- **Outputs**
  - *player_fire*: The signal is set high when the player fires a missile.
  - *flash_effect*: indicates blinking behavior during the player's invincibility phase.
  - *[9:0] player_x*: the player's horizontal position.
  - *[2:0] player_life_count*: three maximum player's life
  - *invincible_flag*: signal is set high when the player is in the invincibility phase.
- **Operation**
  With each rising edge of the "frame_tick" or when a "rst_game" occurs, the module updates the active player's state, such as position, firing status, lives, and whether they are currently in the invincibility phase. At reset, the player at the bottom screen center (x = 288) has three lives, no missiles launched, and is not in the invincibility phase.
  During the gameplay, the module will track the "collected" signal state: if active and the player has less than three lives, one life is added. If the player gets hit ("player_collision" is high), one life will be reduced, and invincibility is activated. While the invincibility phase is active, "invincible_flag" will stay high, toggling "flash_effect" based on the counter. This phase takes 240 frames and ends after that.
  The horizontal position of the spaceship can be changed by the keyboard inputs:
  - "kbd_input" = 4 or press A: the player moves to the left by 6 pixels,
  - "kbd_input" = 7 or press D: the player moves to the right by 6 pixels.

Those movements are within the screen's boundaries (0-576 pixels). To achieve a smooth and clean visual, the movement is controlled by a 12-bit subpixel resolution, "subpixel_position." The “player_fire” is set high for one frame when the spacebar is pressed (“keycode” = 4).

4) **enemy_logic**
   - **Purpose**
     The enemy_logic modules control the enemy behavior, including movement and animation.
   - **Inputs**
     - *frame_tick*: vertical sync signal for the clock.
     - *rst_game*: asynchronous system reset.
   - **Outputs**
     - *[7:0] enemy_anim_frame*: signal for sprites enemy toggling for animation visual.
     - *[9:0] enemy_pos_y*: horizontal offset position of the enemy array.
     - *[9:0] frog_y_offset*: vertical position of the top enemy row.
     - *[9:0] jelly_y_offset*: vertical position of the middle enemy row.
     - *[9:0] other_y_offset*: vertical position of the bottom enemy row.
     - *[9:0] enemy_y_offset*: vertical offset of the entire enemy array.
   - **Operation**
     Whenever the signal “frame_tick” rises or “rst_game” is triggered, the module updates the position and the animation state of the enemy formation. The animation is executed using an 8-bit counter, which enables the “enemy_anim_frame” signal to alternate between two sprite frames every 50-70 frames. A 12-bit “subpixel_offset” register allows for smooth, incremental movement in both directions, vertical and horizontal. The horizontal movement direction is toggled once the defined screed limit (i.e., 0 and 16 pixels) is reached. The entire formation moves down vertically every 120 frames, updating the internal “enemy_y_position” accordingly.
     Every output is calculated from the internal registers: “enemy_pos_y” reflects the enemy's current horizontal position, “enemy_y_offset", "frog_y_offset", “jelly_y_offset", and “other_y_offset” reflect the absolute vertical position of three rows of the enemy, each row occupying 32 pixels. This logic generates the synchronized and controlled enemy’s behavior.

5) **player_missile_ctrl**
   - **Purpose**
     This module manages up to PMAX simultaneous player-fired missiles, handling their creation, position updates, and removal upon collision or exiting the screen.
   - **Parameter**

- *PMAX*: the maximum number of player's missile allowed active in the screen at the given time.
-

- **Inputs**
  - *rst_game*: The asynchronous system is reset to clear the state and all the missiles.
  - *[9:0]_player*: current X or horizontal position of the player
  - *frame_tick*: vertical sync signal for clock in movement updating.
  - *spawn_player_missile*: signal is set high when the spacebar is pressed, triggering the missile creation.
  - *[PMAX-1:0]__missile_hit_flags*: Binary flag vector with size three. This vector tracks the collision of each missile and removes the missile if the collision occurs.
- **Outputs**
  - *[PMAX-1:0]__missile_active*: The bit vector to track the active player missiles.
  - *[9:0]_player_missile_x_[PMAX]*: the X-positions of the active player missiles.
  - *[9:0]_player_missile_y_[PMAX]*: the Y-positions of the active player missiles.
- **Operation**

  The player_missile_ctrl module keeps track of the state of player missiles on each rising edge of "frame_tick" or on "rst_game ."Initially, during reset, all missile slots get cleared and set to their default positions. Then, when the "spawn_player_missile" signal rises (0 becomes 1), the module attempts to find the first available missile slot (i) and creates a new missile by setting "missile_active [i]" to 1, adjusting its X position to the player's current position plus 30 ("player" + 30), and setting its Y coordinate to 448. The "creation_flag" guarantees that missiles are only created once per key press.

  Active missiles ("missile_active [i]" == 1) are set to move upwards 4 pixels with every frame update. A missile is capable of being deactivated by clearing its "missile_active" bit if it reaches the top of the screen (Y == 0) or has the corresponding bit in the "missile_hit_flags" set. PMAX missiles are configured to be active at the same time, and each missile slot has its lifecycle controlled with frame-based synchronization.

6) enemy_missile_ctrl

- **Purpose**

  This module creates and updates the movement of an enemy missile that is fired every two seconds by the enemy column nearest the player's horizontal position. It tracks the missile's vertical movement until it goes off-screen or hits the player.

- **Inputs**
  - *rst_game*: asynchronous system reset to clear the enemy_missile_ctrl and internal state.
  - *[9:0] player_pos_x*: The horizontal position of the player's spaceship aims to choose the nearest enemy column to fire the missile.
  - *[9:0][2:0] enemy_active_flags*: to record the alive status of every enemy (3x10 enemies).
  - *frame_tick*: vertical sync signal for clock logic.
  - *[9:0] enemy_pos_y*: the horizontal position of the enemy array.
  - *[3:0] curr_state*: the current state to ensure that the enemy's missile only fires in the Play state.
  - *[9:0] enemy_y_offset*: the vertical position of the top row enemy.
- **Outputs**
  - *missile_active*: The signal is set high when an active enemy missile is on the screen.
  - *[9:0] enemy_missile_x*: the current horizontal position of the enemy missile.
  - *[9:0] enemy_missile_y*: the current vertical position of the enemy missile.
- **Operation**

  The module is activated on the rising edge of "frame_tick" and functions solely within the "Play" state (curr_state == 4'd1). It fires missiles at a predetermined rate, which it controls using an internal timer ("missile_timer"); a new missile is triggered every 120 frames or roughly every two seconds at 60Hz. The module calculates the missile column based on the player's x-column position by retrieving the highest four bits of "player" to select a "column_index" and determining if a missile can be fired ("exists" == 0). Subsequently, it checks the "enemy_active_flags" array to identify the lowest alive enemy in that column and determines the "enemy_missile_y" starting position (top/mid/bottom row) based on the enemy's position. The missile's horizontal position is set to "enemy_missile_x," which is offset by the global enemy array offset.

  A missile is fired aligned to a specific pixel column and proceeds to descend at a rate of 3 pixels per frame ("enemy_missile_y" += 3). The enemy missile is deactivated by voiding "missile_active" if that missile goes past the screen's bottom limit ("enemy_missile_y" >= 480). All the enemy missile logic is cleared once the game leaves the Play state or the game is reset. This module guarantees that fire from enemies appears dynamic, accurately aimed, and low in frequency and mitigates the effects of multiple overlapping enemy missiles.

7) **detect_collision**
- **Purpose**

  This module will detect collisions in real time between any active player missiles and enemies, the mystery alien, or enemy missiles and the player ship. Once a

collision is detected, the module produces single-cycle pulse flags for each collision. The appropriate responses from the game will occur, e.g., removing game objects, incrementing the score (by how many enemies are destroyed), or decrementing life.

- **Parameter**
  - *PMAX*: the maximum number of player's missiles allowed active on the screen at the given time.
- **Inputs**
  - *rst_game*: Resets all internal flags and state.
  - *frame_tick*: Synchronizes collision check to the VGA frame update.
  - *[PMAX-1:0] player_missile_active*: bits Indicates which player missiles are currently active.
  - *[9:0] player_missile_x*: bits X-positions of player missiles.
  - *[9:0] player_missile_y*: bits Y-positions of player missiles.
  - *[9:0] enemy_missile_x*: X-position of the enemy missile.
  - *[9:0] enemy_missile_y*: Y-position of the enemy missile.
  - *[9:0] player_pos_x*: Left X-position of the player ship.
  - *[9:0] enemy_pos_y*: Horizontal offset of the array of enemies.
  - *[9:0] enemy_y_offset*: Vertical offset of the array of enemies.
  - *[9:0][2:0] enemy_active_flags*: Indicates which enemies are alive by row and column.
  - *mystery_exists*: High if the mystery alien is active.
  - *[9:0]_mysteryX*: X-position of the mystery alien.
  - *[9:0]_mysteryY*: Y-position of the mystery alien.
- **Outputs**
  - *[PMAX-1:0] pmissile_collided*: bits Flags that show which player missiles have struck an object.
  - *[6:0]_enemy_got_hit:* Indicates the index of the enemy that was hit ({column, row}).
  - *player_collision*: Goes high for one cycle when an enemy missile strikes a player.
  - *enem_coll*: Goes high for one cycle when a missile fired by the player hits an enemy.
  - *mystery_hit*: Becomes high for one cycle when a player missile hits the mystery alien.
- **Operation**

  Collusion detection will be performed on every rising edge of "frame_tick" for every active projectile. The module begins with zeroing out the one-cycle pulse flags. For each active player missile (“player_missile_active [i]” == 1), its position will be calculated compared to the enemy array (using “enemy_pos_y” and

"enemy_y_offset"). Based on the normalized Y-coordinates, tests for overlaps against all three enemy rows will be done: top (Y: 0–31), mid (32–63), and bottom (64–95). For each enemy row, the module will also examine whether the enemy row is alive by checking "enemy_active_flags[normX[9:6]][row]" == 1. Once a collision is detected, the "pmissile_collided[i]" will be set to high, the "enem_coll" will be set to high, and the particular index of the enemy that was hit will be encoded into "enemy_got_hit."

The module will also check for collisions between each player missile and the mystery alien using bounding-box overlap (32×32 pixels). If there is an overlap, the "mystery_hit" flag will be raised, and "pmissile_collided[i]" will be set to high.

Finally, it checks whether the enemy missile intersects with the region of the player ship (Y: 452–480 and X: player_pos_x to player_pos_x + 64). If a collision is detected, the "player_collision" will be set high for only a single cycle. All outputs will synchronize with "frame_tick" and are all intended to be one-cycle pulses in order to act as triggers to other subsystems.

8) score_tracker

- **Purpose**
  This module is responsible for tracking score multipliers and showing the player's score in real time using BCD (Binary-Coded Decimal) logic. Each collision event near the player will be turned into a frame-synchronous event via the frame clock and updated symbolically as a pixel- and index-based number displayed on the VGA pixel.
- **Inputs**
  - *frame_tick*: Vertical sync; the timing anchor for the entire system.
  - *rst_game*: Synchronous reset; resets score and internal state.
  - *[5:0]_X*: Pixel X-coordinate (for selecting and indexing digits).
  - *[3:0]_Y*: Pixel Y-coordinate (for selecting and indexing digits).
  - enemy_collision_flags: One-cycle pulse indicating that an enemy has been collided with and hit.
  - *[2:0]__enemy_row*: Indicates what row the collision with that enemy occurred in (for scoring point weighting).
  - *mystery_hit*: One-cycle pulse indicating that the mystery alien has been collided with and hit.
- **Outputs**
  - *score_pixel_data*: The value of a binary pixel (0 or 1) for conveying to VGA.
- **Operation**

Upon every rising edge of “frame_tick” or during “rst_game," the module will process scoring events and update the visually displayed score as necessary. To synchronize the asynchronous “enemy_collision_flags," two flip-flops (“col_sync_0”, “col_sync_1”) will be used and form a clean rising edge to dictate an event has occurred (“col_edge”). When either “col_edge” or “mystery_hit” is high, the module will establish a score increment (“delta”) based on the row of the enemy: top = +3, middle = +2, bottom = +1, +10 for mystery alien collisions.

The score is kept in three 4-bit BCD digits: “score_hundreds," “score_tens," and “score_ones." When a scoring event occurs, the digits will be updated using addition with carries where appropriate, taking care to wrap at 10 for every digit.

For rendering, the “X” input will determine which digit (hundreds, tens, ones) is tied to the transaction. Each BCD digit will select the base address set in the “number_rom," and each “Y” coordinate will be the row index into the ROM to retrieve an 8-bit slice from the ROM. The “score_pixel_data”=slice pixel for output. This ensures the creation of a real-time rendering method for managing a three-digit score value using font ROM data mapped to a VGA screen.

9) **mystery_controller**

- **Purpose**
  This module is responsible for controlling the behavior of the mystery alien, including the random horizontal spawn direction, if it has traversed the screen, if it is visible, and if it has been successfully hit by a player missile to trigger a life-drop event.
- **Inputs**
  - *frame_tick*: vertical sync signal and clock for the module.
  - *rst_game*: asynchronous reset to set internal state and counters.
  - *enemy_y_offset*: vertical position of the main enemy block; indicates that mystery logic can proceed after descent.
  - *mystery_hit*: pulse indicating the mystery alien was hit by a missile.
- **Outputs**
  - *missile_active*: High when the mystery alien is active and visible on screen.
  - *spawn_life*: High for one cycle to signal to spawn a life-drop object.
  - [9:0]*x_pos*: Current horizontal position of the mystery alien.
  - *[9:0]_y_pos*: Current vertical position of the mystery alien.
  - *[9:0]_mystery_y_out*: Vertical position output for use by rendering modules.
  - *[9:0]_life_x_pos*: x-position of the life drop when spawned.
  - *[9:0]_life_y_pos*: y-position of the life drop when spawned (fixed).
- **Operation**

Every rising edge of “frame_tick” or “rst_game," the module runs through the mystery alien behavior and updates the relevant internal state. On a “rst_game” (asserted) or if the enemies have yet to descend (“enemy_y_offset” < 64), the module clears all internal variables and turns off the mystery alien. Otherwise, when rst_game (de-asserted), it will increment a “spawn_timer” every frame up to the SPAWN_T = 119 value. When the spawn_timer equals SPAWN_T, the mystery alien becomes active by asserting exists, and a direction (dir) and starting X-position (player_x) determined using a 4-bit linear feedback shift register (LFSR); this random bit was then used to select left-to-right or right-to-left travel.

When active, the alien will move horizontally at a fixed speed (SPEED = 1) and is removed (“missile_active” <= 0) when it traverses outside the screen boundary. If a “mystery_hit” occurs while active, the module will assert “spawn_life” for one frame and retain the current X-position as the drop point (“life_x_pos”) and a fixed Y-position (“life_y_pos” = Y_START), allowing the downstream life-drop system to enable as needed.

The mystery alien will always appear at the same Y-position (Y_START = 32), and both timing and visibility logic must be synchronized to the VGA frame rate. Hence, the animation looks smooth, and collisions register appropriately.

10) life_drop_controller

- **Purpose**
  This module controls the falling life bonus that appears after the mystery alien is hit. It provides control over life drop position, movement, visibility, and collision with the player and provides a pulse to signal a successful collection.
- **Inputs**
  - *frame_tick*: Frame sync signal for update timing.
  - *rst_game*: Asynchronous reset for internal states.
  - *spawn_life*: One cycle signal that a life drop should start falling.
  - *[9:0] spawn_x*: X coordinate at which the life drop will start falling (from the mystery alien).
  - *[9:0] spawn_y*: Y coordinates at which the life drop will start falling. [9:0] ship_x: X position of the player ship (used for collision detection).
  - *[9:0] ship_y*: Y position of player ship.
- **Outputs**
  - *[9:0] life_x*: Current X position of falling life icon.
  - *[9:0] life_y*: Current Y position of falling life icon.
  - *life_visible*: High when the life icon is on the screen and still falling.
  - *collected*: High for two cycles when the player collects the life bonus.

- **Operation**
  At each rising edge of “frame_tick” or when “rst_game” is asserted, the module updates the life drop state. When spawn_life is asserted (usually after hitting the mystery alien), the module initializes the life drop position from inputs “spawn_x" and “spawn_y." It sets “falling” to 1, making the life icon visible, and starts it on its way down.

  While “falling” is asserted, the icon moves one pixel down each frame (drop_y += SPEED). During falling, the module checks for overlap with the player ship by bounding-box collision detection. When the drop overlaps the player ship's position, the falling flag is cleared, and “collect_counter” is set to 2, resulting in a 2-cycle collected pulse.

  If the drop reaches the bottom of the screen (drop_y >= SCREEN_HEIGHT), it is deactivated and not collected. At each frame, the “collect_counter” is decremented, ensuring a clean, short collection signal. The outputs “life_x," “life_y," and “life_visible" give the current output related to the drop, and collected provides a timing-safe pulse to sync with the player life system.

3.2 Output and Display Logic

11) Color_Mapper

- **Purpose**
  This module calculates the RGB color output for each pixel (within pixel clock resolution). For each pixel, the draw module computes the list of all visible game entities by evaluating the game state, game entity's position, and sprite data for each. The following game entities are visible on the VGA display: player, enemies, missiles, mystery alien, remaining player lives, score text, and the background image.
- **Parameter**
  - *PMAX*: The maximum number of player missiles that can exist on screen at the same time.
- **Inputs**
  - *[3:0] curr_state*: Current state of the game (title screen, gameplay, win/loss).
  - *[9:0] DrawX*, *[9:0] DrawY*: Current pixel position supplied from the VGA controller.
  - *lose_text_pixel*, *win_text_pixel*, *start_text_pixel*: Control signals for rendering text elements depending on game state.
  - *invincible_flag*, *flash_effect*: Control signals to draw the player's visual status.

- *[9:0] enemy_pos_y*, *enemy_y_offset*: Top-left coordinate of the enemy sprite array.
- *[9:0][2:0] enemy_active_flags*: Status for each enemy (alive/dead per row
- *[7:0] enemy_anim_frame*: Only used to cycle through enemy sprites (animation).
- *[9:0] enemy_missile_active*, *[9:0] enemy_missile_x*, *[9:0] missileY*: Existence and position of enemy missiles.
- *[PMAX-1:0] player_missile_active*, *[9:0] player_missile_x [PMAX]*, *[9:0] player_missile_y [PMAX]*: : Existence and position of player missiles.
- *[9:0] player_offset*: X-coordinate of the player's ship.
- *[2:0] life_count*: Player's lives (0 to 3).
- *score_label_pixel*, *score_num_pixel*: Control signals to render score.
- *lives_icon_pixel*: Control signal to render the text "Lives."
- *[9:0] mystery_exist*, *[9:0] mysteryX*, *[9:0] mysteryY*: Existence and position of the mystery alien.
- *[9:0] life_x*, *[9:0] life_y*, *life_visible*: Coordinates and visibility of the falling life icon.

- **Outputs**
  - *[7:0] VGA_R*, *[7:0] VGA_G*, *[7:0] VGA_B*: Final red, green, and blue outputs for the VGA display.
- **Operation**

  This module runs on every pixel clock. It collects the current coordinates (“DrawX," “DrawY”) and then applies its game logic to determine what object is placed at the pixel, if any. The decision tree for whether or not to draw objects is based on the game state, enabling conditions for each entity, and the normalized sprite coordinates.

  The background and transparent area will display the default gradient blue color. For every successive frame:
  - **Text rendering**: Any active game messages, such as "Space Invaders Pust Start," "You Win," "Game Over," etc., will be rendered.
  - **HUD elements**: The current number of lives remaining and the score will be drawn from the ROM bitmap data.
  - **Player ship**: The player ship will be drawn from “ship_rom” when it does not go through visual invincibility flashing.
  - **Enemies**: The three rows of enemies will be drawn using row visibility, multi-layer ROM slices, and animation offsets. The enemies' color encoding also uses multi-colored sprites, allowing for 2-bits per pixel color encoding.

- **Missiles**: The enemy and player missiles will be drawn (either solid white or red colored) at their coordinates when active.
- **Mystery alien**: If the mystery ship is enabled, it will be drawn using its 2-bit-per-pixel ROM slice and color mapping.
- **Life Drop**: If a heart is enabled and visible, it will be drawn using an 8×8 heart icon and in yellow color at its coordinates.

Each draw block will include conditional logic to evaluate whether the current pixel is in the position of a particular object. It will then retrieve the correct sprite data from ROM, extract the pixel-based color encoding of the object being drawn, and update the corresponding output red, green, and blue color. Based on draw order priority, only one entity will be drawn for each pixel.

12) VGA_controller

- **Purpose**
  To provide all the synchronization signals and pixel coordinates to drive a standard 640×480 VGA display with precise VGA timing.
- **Inputs**
  - *Clk*: 50 MHz. Active system clock (not used internally in this module).
  - *Reset*: An active high signal is used to reset all the counters and outputs to a known state.
  - *VGA_CLK*: 25 MHz. The pixel clock is used to drive the VGA timing logic.
- **Outputs**
  - *VGA_HS*: Horizontal sync signal (active low).
  - *VGA_VS*: Vertical sync signal (active low).
  - *VGA_BLANK_N*: Indicates if the pixel being generated is in the visible area (active high).
  - *VGA_SYNC_N*: A constant logic-low signal for the VGA DAC is required.
  - *[9:0] DrawX*: The current horizontal pixel coordinate (0–639 active range).
  - *[9:0] DrawY*: The current vertical pixel coordinate (0–479 active range).
- **Operation**
  The controller uses two internal counters, “h_counter” and “v_counter," to keep track of the horizontal and vertical pixel positions in the VGA frame, which is a total of 800×525 pixels. The counters count in active state on the rising edge of the “VGA_CLK ."The “h_counter” reaches its “H_TOTAL” and resets to zero, and increments “v_counter." A similar operation occurs with “v_counter” as it reaches “V_TOTAL” before also resetting to zero. The horizontal and vertical sync signals (“VGA_HS," “VGA_VS”) will be asserted low during the horizontal synchronization window (656–752) and the vertical synchronization window (490–492). The “VGA_BLANK_N” output is asserted high if the current pixel is not in view (X < 640 and Y < 480); otherwise, it is asserted low. The Pixel Raster is

continually updated to “DrawX” and “DrawY” registers whenever the “h_counter” and “v_counter” output is in their active ranges. All signal control outputs are registered on the rising edge of VGA_CLK for stable timing.

13) vga_clk

- **Purpose**
  To generate a stable 25 MHz VGA pixel clock (*c0*) using an Altera Phase-Locked Loop (PLL) malfunction from a 50 MHz reference clock (*inclk0*).
- **Inputs**
  - *inclk0*: 50 MHz reference clock input to the PLL.
- **Outputs**
  - *c0*: 25 MHz output clock for a VGA pixel clock.
- **Operation**
  The *vga_clk* module implements an Altera still mega function configured in normal mode to scale down the 50 MHz reference clock to a 25 MHz reference clock. The PLL functions are to multiply the reference clock by 1 and divide it by 2 internally, maintaining the 50% duty cycle. The output clock signal *c0* generated by the PLL serves as the pixel clock signal for VGA timing to maintain the clock timing for a 640×480 video display. The *vga_clk* module is implemented in a closed-loop mode, which does not require dynamic reconfiguration or feedback adjustments. All unused ports were terminated for simplicity and stability.

14) HexDriver

- **Purpose**
  This module converts 4-bit binary inputs to the respective 7-bit code that drives a seven-segment display according to the hexadecimal digits (0-F).
- **Inputs**
  - *In0*: Represents the binary value that will be converted to a hexadecimal digit for display.
- **Outputs**
  - *Out0*: Active-low seven-segment encoding representing the input digit.
- **Operation**
  The module continuously observes the 4-bit input *In0* and processes each valid hex value (0 to F) to drive the respective values on *Out0* using a case statement. Each bit in "Out0" corresponds to one of the seven segments of the display, except logic low (0), which is used to signify an active segment. The design is a *unique case* structure suitable for efficient synthesis, and it includes a *default case* that indicates unknown output while the input is invalid. This module serves as a way of rendering hexadecimal digits on standard seven-segment displays in digital applications.

4. Design Procedure / State Diagram / Simulation Waveform

4.1 Overview of the Design Procedure

4.1.1 Foundation Project or Base Code

This project was developed on top of the foundation provided in ECE 385 **Lab 8: SoC with USB and VGA Interface in SystemVerilog**. Lab 8 provided a working base for VGA display control and USB keyboard interfacing using the CY7C67200 USB controller. It also provided a reference for VGA pipeline integration, USB input via NIOS II, and hardware-software synchronization.

Several of the SystemVerilog modules from Lab 8 were reused directly or slightly modified for this game. The files *HexDriver.sv*, *vga_clk.v*, and *VGA_controller.sv* were reused with no modifications because of their stability and generic nature. Other files like *Color_Mapper.sv*, *hpi_io_intf.sv*, and *lab8.sv* were modified and expanded upon to accommodate additional gameplay features like sprite rendering, entity prioritization, and frame-synchronous interactions during gameplay.

The decision to reuse Lab 8 code was based on reasonableness. The lab manual even notes how time-consuming and prone to errors it is to implement USB and VGA subsystems. Rather than developing from a totally blank slate, integrating this tested infrastructure allowed us to spend more time on higher-level gameplay design, modular integration, and interactive features, leaving the USB polling and VGA timing intact.

This reuse offered a reliable base that helped enable the development of a fully playable, real-time Space Invaders game on actual hardware.

4.1.2 Project Objectives and Design Choices

The principal aim of this project is to design a real-time hardware-based Space Invaders game on the DE2-115 FPGA using SystemVerilog and C. This design consists of fully integrated USB keyboard inputs, sprite-based VGA rendering, and modular game logic based on a finite-state machine. Below are the primary design choices made:

- Inputs:
    - A USB keyboard via CY7C67200 controller is utilized to control player actions (move, fire).
    - The NIOS II processor receives keycodes via on-chip I/O and transfers information to hardware through memory-mapped I/O.

- State Machine:
    - A four-state FSM is implemented to handle all game states (1. Start, 2. Play, 3. Win, 4. Lose).
    - The state transition is based on the player's remaining lives, enemy status, mystery alien status, and keyboard input.
- Sprites:
    - Every entity in the game (player, enemies, missiles, mystery alien, and UI text) has been implemented as pixel-level sprites stored in on-chip ROM.
    - In each frame, entities are rendered conditionally by checking position and visibility flags.
- Algorithms:
    - Enemy AI follows the players' horizontal position via column proximity.
    - For each frame, collision detection is evaluated by implementing bounding box logic.
    - Additional features, such as mystery alien spawn, life drop and abductions, and invulnerability features, are implemented via FSMs and counters.
- Storage Units:
    - The sprites are stored in on-chip ROM (SystemVerilog).
    - Keycodes and other signals are stored and transferred to hardware via registers or wires between modules.
    - The C code, USB driver, and data structures are stored in on-chip memory2_0.
- Outputs:
    - The game visuals are rendered to a VGA display using a custom color-mapped pipeline.
    - The score and player lives are shown either as bitmap digits or as icons.
    - The hex displays can show the current keycode for troubleshooting.
- Platform Designer IPs:
    - *nios2_gen2_0*: NIOS II softcore processor, which handles USB polling and USB keyboard interfacing.
    - *onchip_memory2_0*: This is used to store my C program, stack, and C variables.
    - The *sdram* and *sdram_pll* provide additional external memory interfaces.
    - The PIO modules control all USB control (*usb_hpi_address, usb_hpi_data, usb_hpi_r, usb_hpi_w, usb_hpi_cs, usb_hpi_reset*) and handle keycodes.

This is a modular and synchronized design, allowing for smooth gameplay and quick reaction times while keeping the VGA display flicker-free and the gameplay logic consistent from frame to frame.

4.1.3 Supporting Research and Background Study

Many aspects of background research were performed to build a working and reliable hardware-based Space Invaders game. These areas of research revolve around specific implementation requirements and design considerations that were explored throughout the development process.

- **Understanding USB HID and the CY7C67200 Controller**
  The research commenced by understanding the USB Human Interface Device (HID) protocol itself, which is essential in determining the structure of keyboard inputs and how they are sent. Then, after a comfortable understanding of the HID protocol, the next step was to understand the CY7C67200 USB controller that we were using in our project. This required investigating the Host Port Interface (HPI) protocol, as well as the register map associated with the CY7C67200. This initial stage was essential to set up the Platform Designer interface, write C code to poll keycodes, and later confirm that the data coming in from the USB Keyboard would be correctly sent to the game logic.

- **Reviewing VGA Timing for Pixel-Level Rendering**
  In order for a game to display graphics correctly, various VGA timing diagrams were studied to understand the important sync pulse generation, active game duration, and blanking period durations. This helped in the design of sprite rendering logic and in determining where to use pixel scan timing. There was also valuable information about synchronizing sprite rendering updates with the vertical sync signal, which, from course content and lab documentation, provided essential means to provide an output without flickering and a stable frame.

- **Finite State Machine (FSM) Design for Game Flow**
  To manage game processing and the management of timed events, we revisited ideas of FSM design in lecture materials. We specifically focused ideas on how to define state transitions and the timing of frame-based signal triggers. This area of research is focused on designing a sound and scalable FSM design for events, including Play state, Win state, and Lose state, and also managing features such as missile cool-down times, invincibility windows, and life drops.

- **Sprite Memory Encoding and Layer Composition**
  The creation of different game entities with overlapping positions required research on how to encode and prioritize sprite information. In particular, research on multi-layer rendering, color encoding, and conditional display are conducted. This information helped inform the definitions for sprite ROMs and implemented logic that determines which game entity is displayed at each pixel of a display based on priority and enables flags for each pixel.

- **Avalon-MM and Platform Designer IP Inclusion**
  Since the game design would utilize both NIOS II and Platform Designer components, we examined the documentation associated with Avalon-MM protocols, memory-mapped inputs and outputs, and how to document programmable I/O (PIO) interfaces. Each was used to create a custom memory map to connect C Software with hardware signals such as keycode registers and USB control lines, enabling reliable interfaces and communication of the software and hardware design modules.

- **Frame-Based Timing**
  In aligning the display updates with our gameplay mechanics, we experimented with a combination of frame-based counters and timing coordination with *frame_tick*. Based on the methods previously used in the lab assignments, we adapted timing strategies to regulate the pacing of animations, controls on event delays, and the drawing delay on spawn with a stable frame alignment.

Overall, the design was driven by targeted research to address specific implementation challenges. Each research topic was justified by functional specifications, which had consequences for decisions relating to hardware and software development.

4.1.4 Integration of Design Components

The project objectives related to input handling, game state control, sprite rendering, collision detection, and scorekeeping are functionally unified by shared signals and timing and memory-mapped input and output. Each subsystem interacts with others through specific variables and control paths to provide the appearance of an integrated system.

The *frame_tick* signal is the global timing reference. It synchronizes updates across subsystems, such as missile movement, animation timing, mystery alien spawning timing, and any frame-based counters. Thus, all time-sensitive logic will be kept in time with the display's refresh rate.

State flags such as spawn_player_missile, *missile_hit_flags*, *spawn_life*, and *mystery_hit* propagate back and forth between control modules and rendering or scoring units. These signals were used to coordinate a response to such events as a successful hit, the destruction of the enemy, and possible life drops.

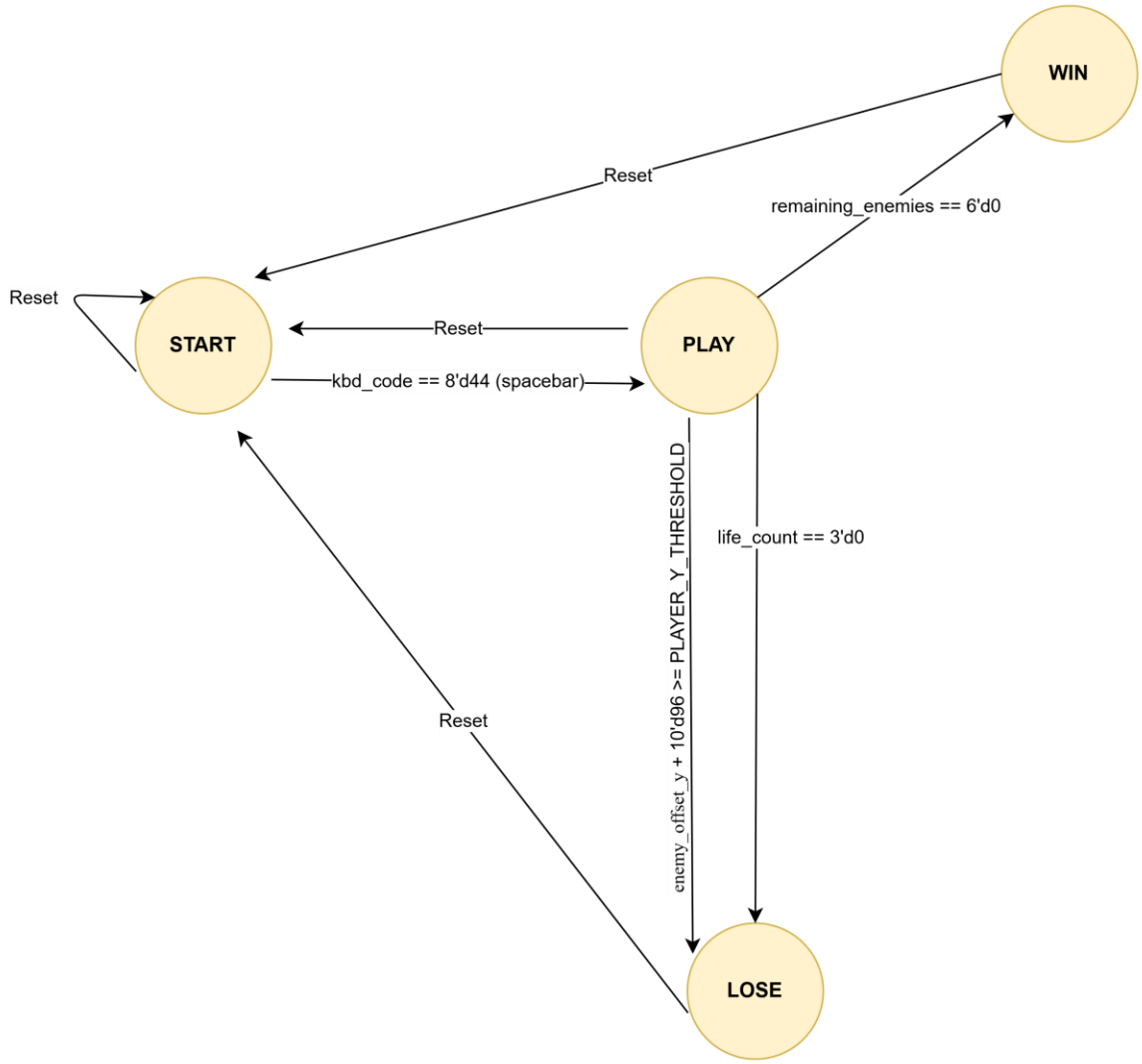
The NIOS II processor updates the keycode register, which connects the USB keyboard input to the hardware logic. The game modules use this register information to trigger actions such as moving a player around the screen, firing a projectile, or restarting the game.

The shared position values such as player_pos_x, player_x, and y_pos help the modules synchronize the render to the logic decisions made using the enable flags to determine what the

sprites were shown on a per pixel. The rendering logic then uses these inputs to determine what entities overlap and display graphics based on priority.

By working together with these shared variables and signals, each project is working toward the common goal of creating a responsive game system based on hardware.

4.2 State Machine



Control logic is implemented as a finite state machine (FSM) with four states: START, PLAY, WIN, and LOSE. It switches between the states based on user input and game status, synchronized with the frame_tick signal for visual consistency.

States and Transitions

- **START (State 0)**

The initial state after reset shows "Press Space" until the spacebar is pressed (kbd_code = 8'd44).

- **PLAY (State 1)**
  Main gameplay state. The player will interact with the game until the player either wins or loses.
- **WIN (State 2)**
  This state is reached when all of the enemies are defeated (remaining_enemies == 6'd0).
- **LOSE (State 3)**
  This state is reached as soon as either the player loses all of their lives (life_count == 3'd0) or enemies touch the bottom of the screen (enemy_y_offset + 10'd96 >= PLAYER_Y_THRESHOLD).
- **Reset Transition**
  A global reset returns an FSM to the START state, no matter the current state.

Justification for No Waveform

This FSM is simple, with well-defined transitions and real-time behavior, which was observed through VGA output. All functionality was tested using hardware; thus, a simulation waveform was not needed.

5. Block Diagram

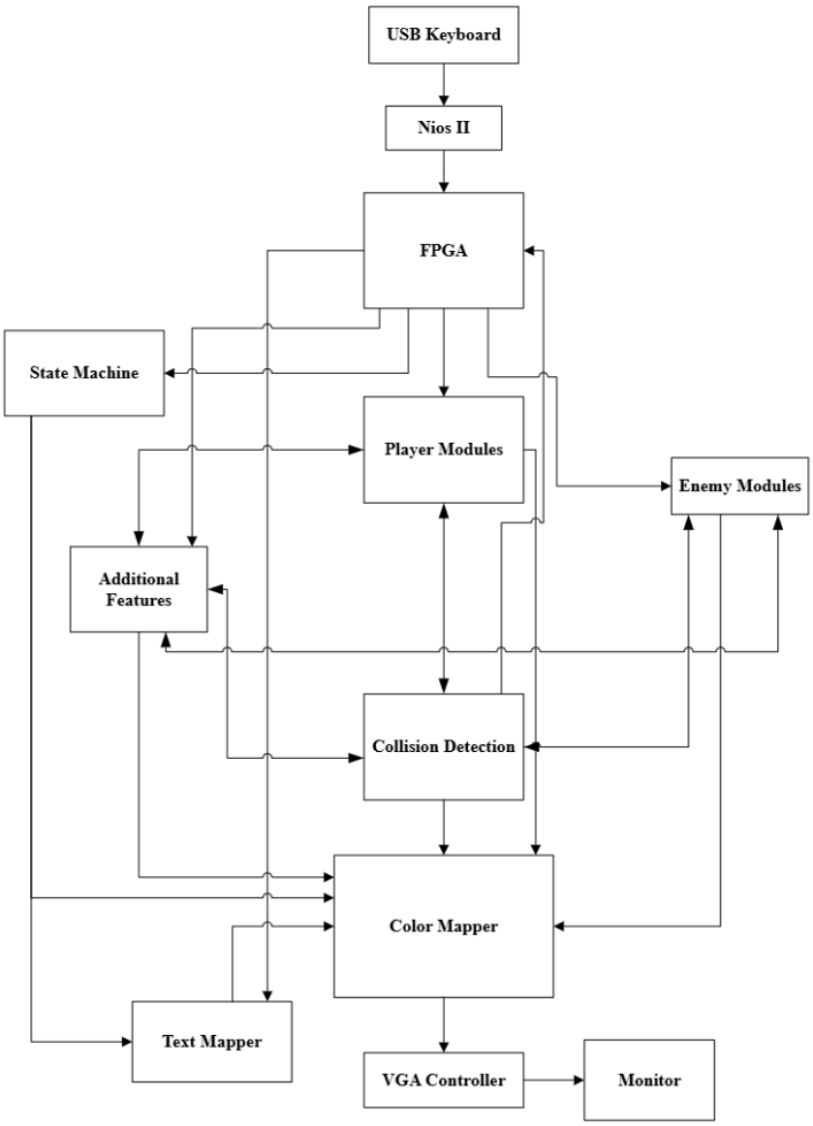


Figure 1 Overview

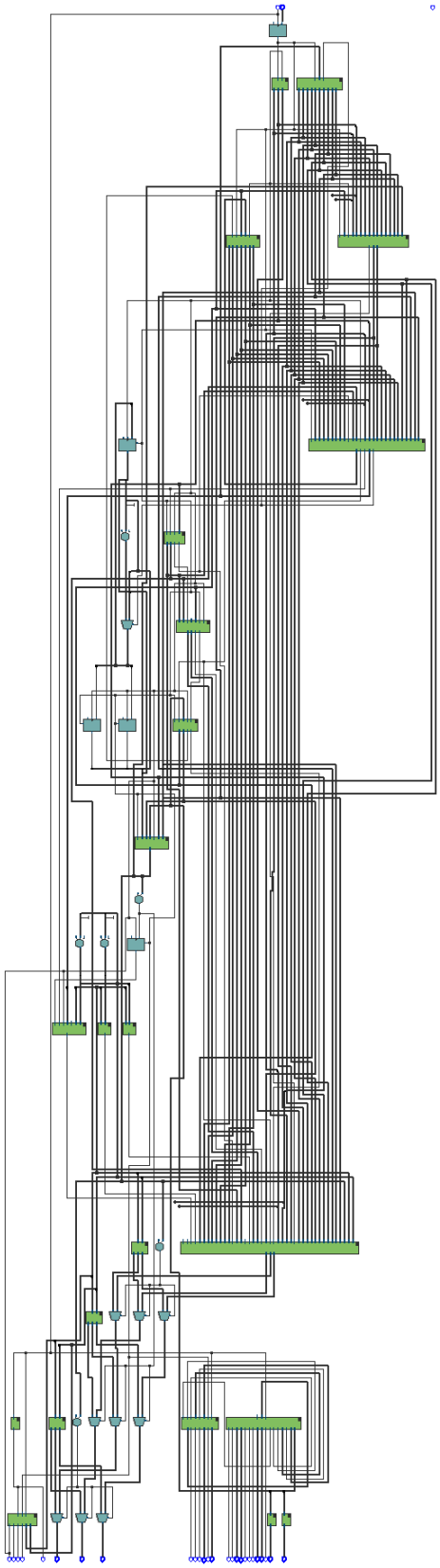


Figure 2 top_level_space_invaders

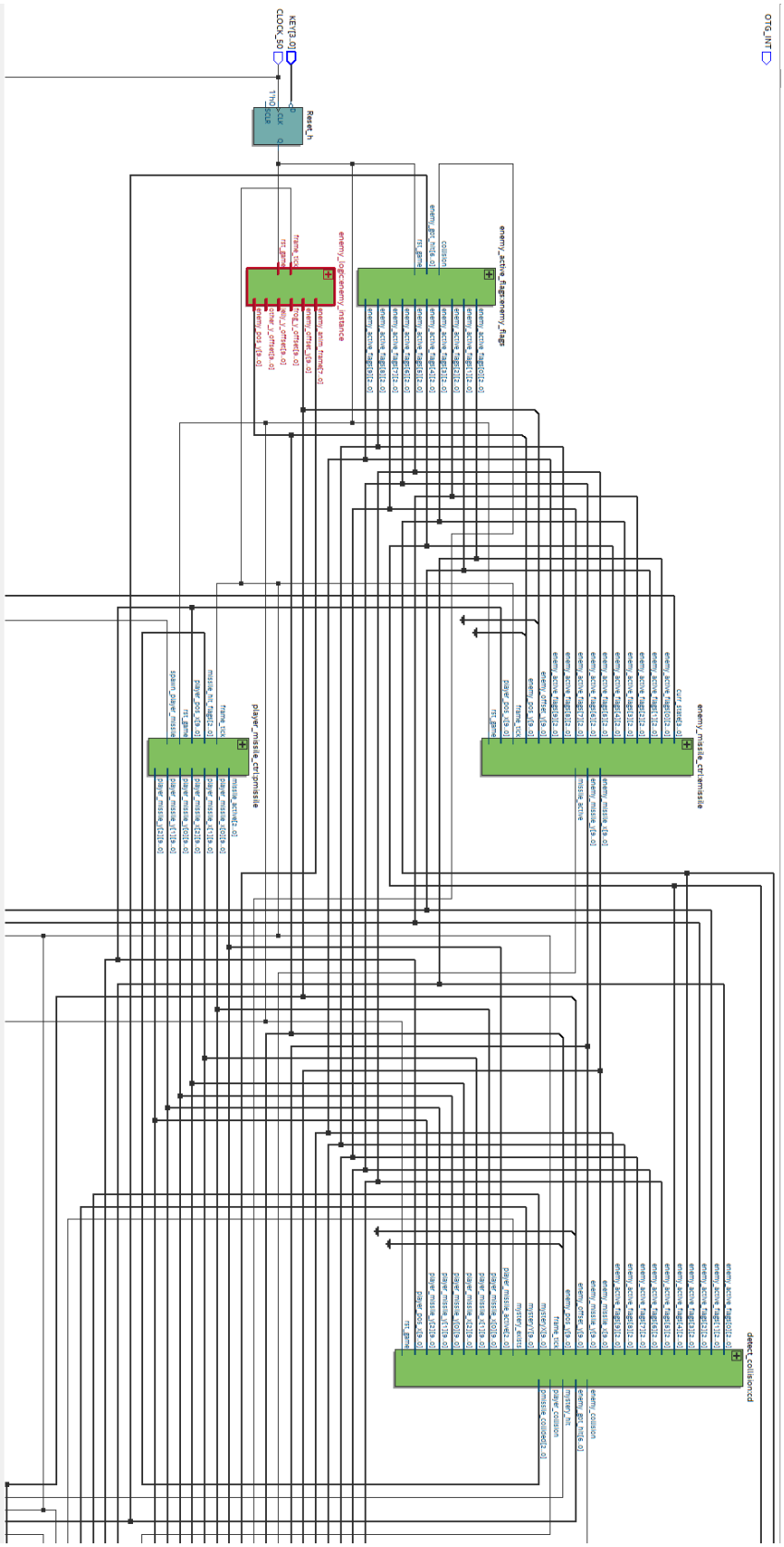


Figure 3 Left Part Zoom In

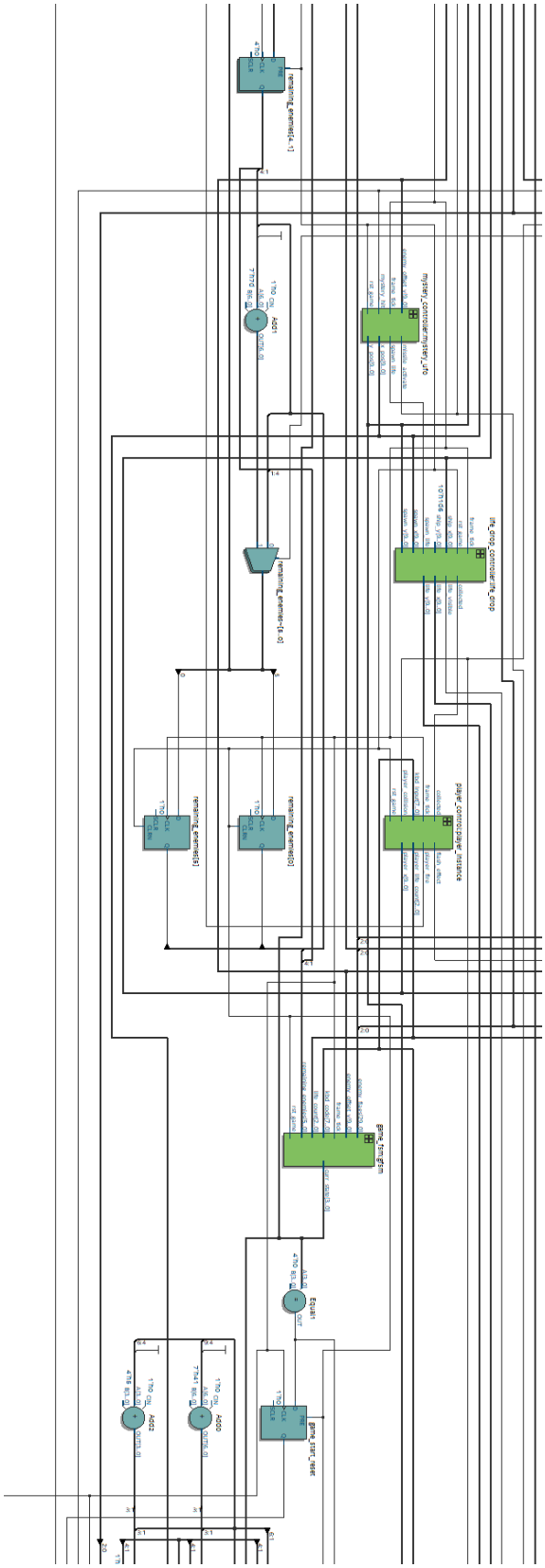


Figure 4 Middle Part Left Zoom In

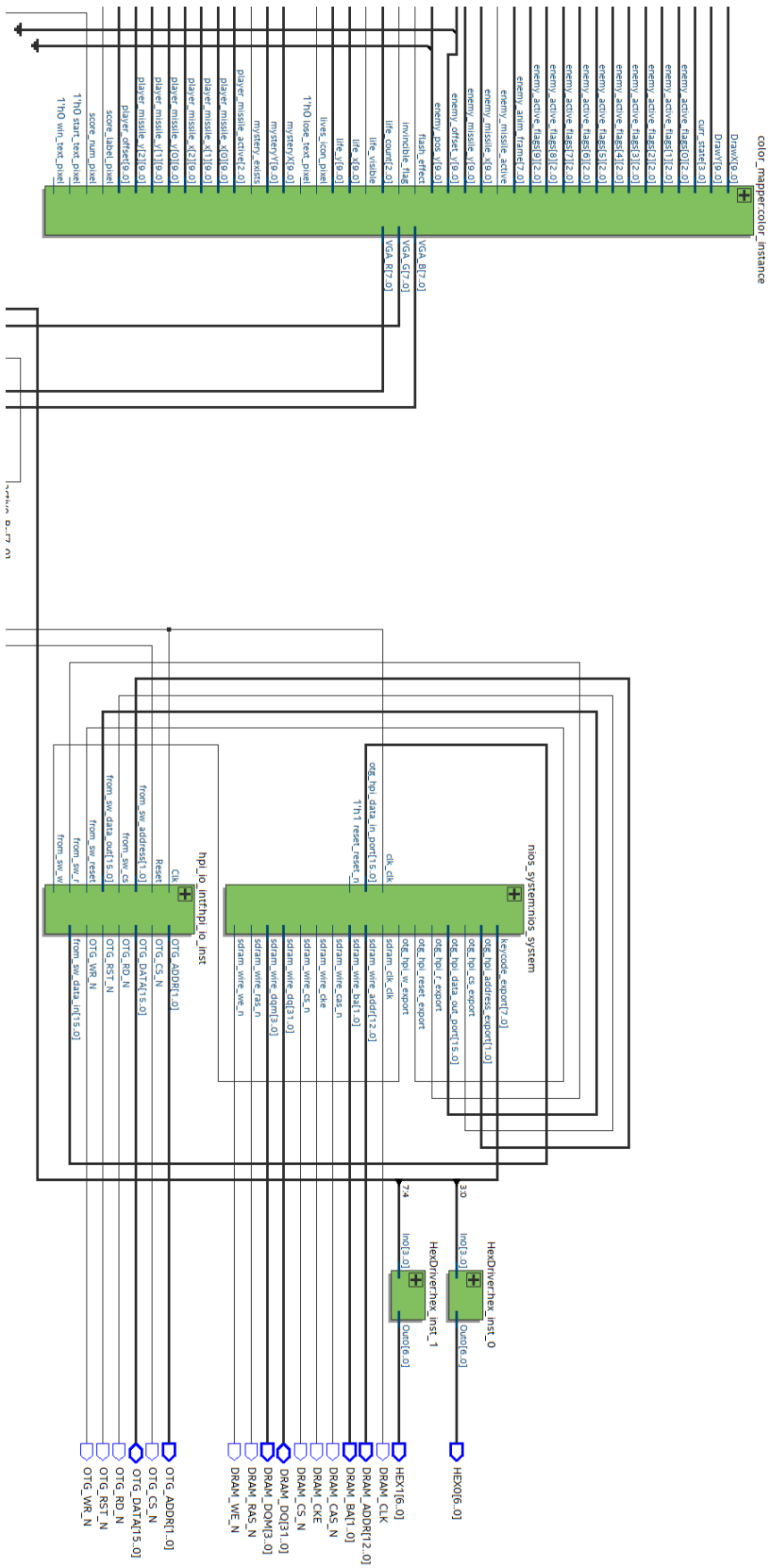


Figure 5 Top Right Zoom In

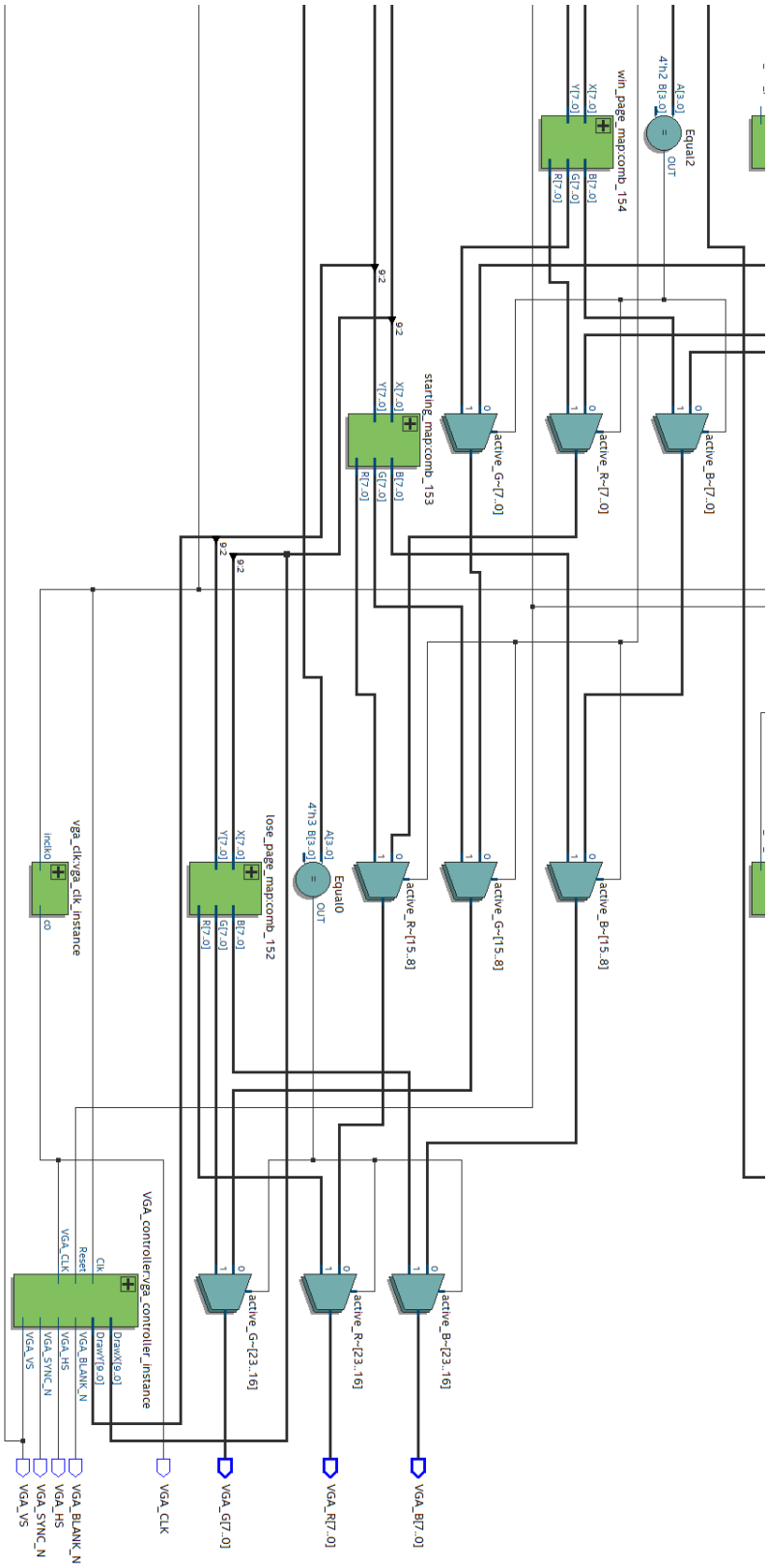
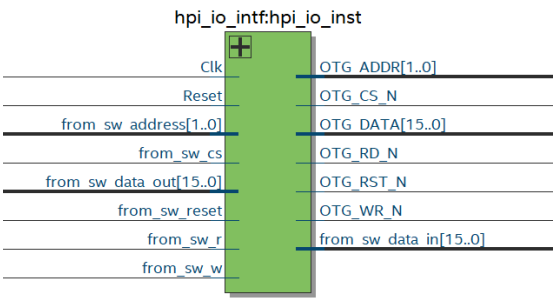


Figure 6 Bottom Right Zoom In

6. SV CODE

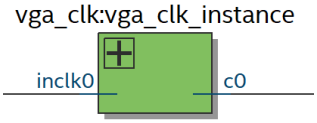
Module: top_level_space_invaders
Input: CLOCK_50, [3:0] KEY, OTG_INT, [15:0] OTG_DATA, [31:0] DRAM_DQ
Output: [6:0] HEX0, [6:0] HEX1, [7:0] VGA_R, [7:0] VGA_G, [7:0] VGA_B, VGA_CLK, VGA_SYNC_N, VGA_BLANK_N, VGA_VS, VGA_HS, [1:0] OTG_ADDR, OTG_CS_N, OTG_RD_N, OTG_WR_N, OTG_RST_N, [12:0] DRAM_ADDR, [1:0] DRAM_BA, [3:0] DRAM_DQM, DRAM_RAS_N, DRAM_CAS_N, DRAM_CKE, DRAM_WE_N, DRAM_CS_N, DRAM_CLK
Description: This is the top-level module that integrates all major components of the system and manages the overall interconnection between them. It serves purely as a structural wrapper and does not implement any processing logic itself. Refer to the block diagram in the report for a visual overview of how the submodules are organized and interact within the design.
Purpose: To be the top module that connects all the input and output of all other module inside the project.

Module: hpi_io_intf



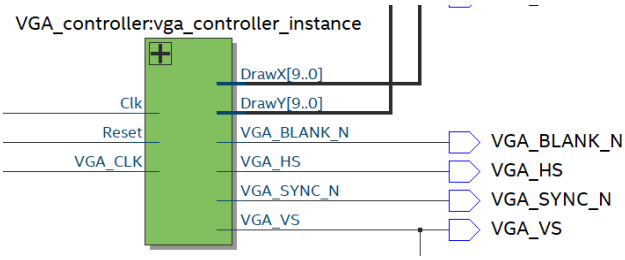
Input: Clk, Reset, [1:0] from_sw_address, [15:0] from_sw_data_out, from_sw_r, from_sw_w, from_sw_cs, [15:0] OTG_DATA
Output: [15:0] from_sw_data_in, [1:0] OTG_ADDR, OTG_RD_N, OTG_WR_N, OTG_CS_N, OTG_RST_N
Description: This module connects the NIOS II processor to the EZ-OTG USB interface chip by mapping the processor's control, address, and data signals to the corresponding OTG bus lines. It buffers outgoing data, manages the bidirectional data bus using a tristate driver, and ensures that signal directions and values are properly synchronized.
Purpose: Its purpose is to serve as a hardware-level communication bridge between the processor and the USB interface, enabling data transactions without implementing any processing logic itself.

Module: vga_clk



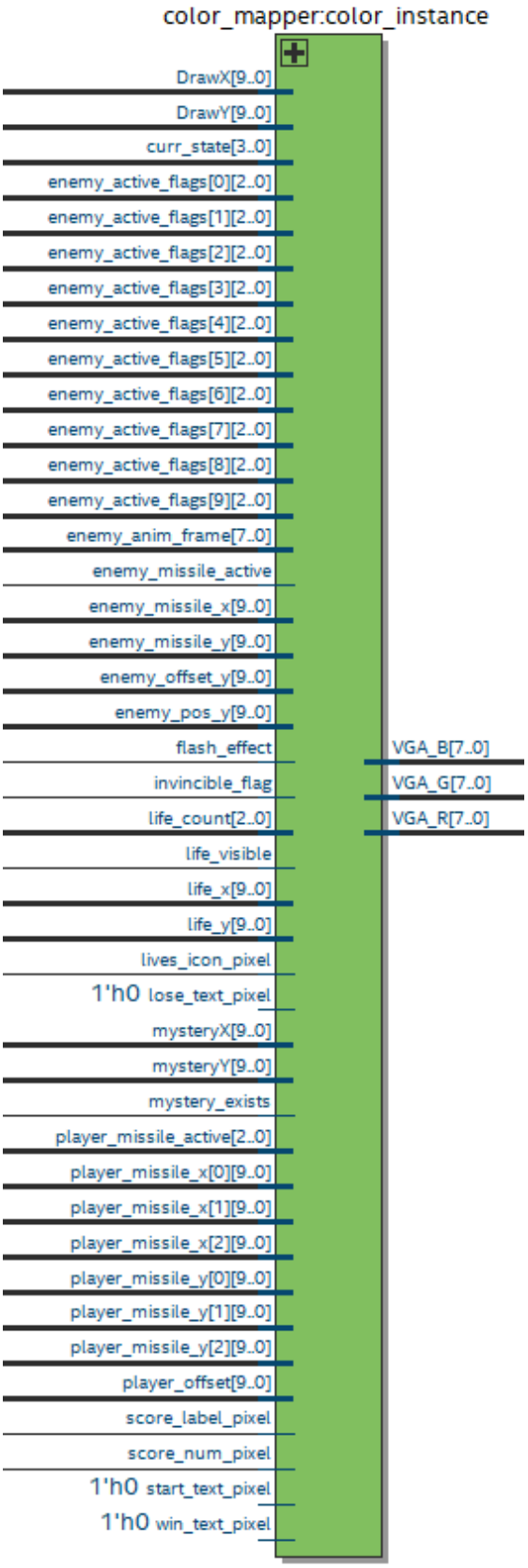
Input: inclk0
Output: c0
Description: This module uses a Phase-Locked Loop (PLL) to generate a 25 MHz clock (c0) from a 50 MHz input clock (inclk0), based on preset frequency parameters. The generated clock is intended for use in VGA display timing, where precise pixel clocking is required.
Purpose: Its purpose is to provide the correctly scaled and synchronized clock signal needed to drive the VGA controller and ensure proper video output timing.

Module: VGA_controller



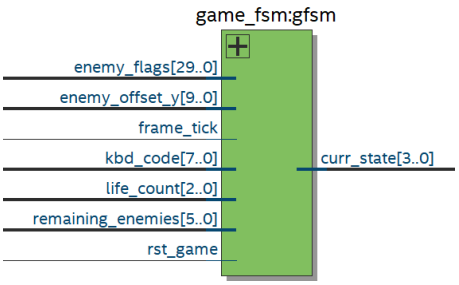
Input: Clk, Reset, VGA_CLK
Output: VGA_HS, VGA_VS, VGA_BLANK_N, VGA_SYNC_N, [9:0] DrawX, [9:0] DrawY
Description: This module generates the necessary VGA synchronization signals (VGA_HS, VGA_VS, VGA_BLANK_N) and current pixel coordinates (DrawX, DrawY) to control a 640x480 VGA display using a 25 MHz pixel clock. It tracks the horizontal and vertical scan positions using counters and produces sync pulses and blanking signals according to VGA timing specifications.
Purpose: Its purpose is to control the video frame scan sequence and provide display timing signals required for standard VGA output.

Module: HexDriver
Input: [3:0] In0
Output: [6:0] Out0
Description: This module takes a 4-bit binary input and outputs a corresponding 7-bit code to drive a seven-segment display, allowing hexadecimal values (0–F) to be visually represented. It uses a combinational logic block with a case statement to assign the correct output for each input value.
Purpose: Its purpose is to convert binary values into human-readable digits on a seven-segment display, commonly used for debugging or displaying numeric outputs.



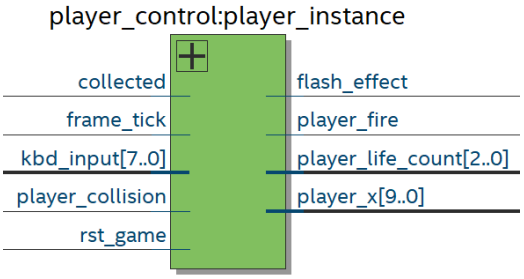
Module: Color_Mapper
Input: [3:0] curr_state, lose_text_pixel, win_text_pixel, start_text_pixel, invincible_flag, [9:0] DrawX, [9:0] DrawY, [7:0] enemy_anim_frame, [9:0] enemy_pos_y, mystery_exists, [9:0] mystery, [9:0] mystery, [9:0] enemy_offset_y, [9:0] player_offset, flash_effect, enemy_missile_active, [9:0] enemy_missile_x, [9:0] enemy_missile_y, [9:0] life_x, [9:0] life_y, life_visible, [PMAX-1:0] player_missile_active, [2:0] life_count, [9:0][2:0] enemy_active_flags, score_num_pixel, score_label_pixel, lives_icon_pixel
Output: [7:0] VGA_R, [7:0] VGA_G, [7:0] VGA_B
Description: This module determines the color output for each pixel on the screen based on the current game state, sprite locations, and draw coordinates. It reads from multiple sprite ROMs and uses internal logic to decide what should be drawn at each pixel, such as the player, enemies, missiles, UI elements, and text.
Purpose: Its purpose is to map game logic and visual elements into RGB values for VGA output, enabling the game's graphics to be displayed on screen.

Module: game_fsm



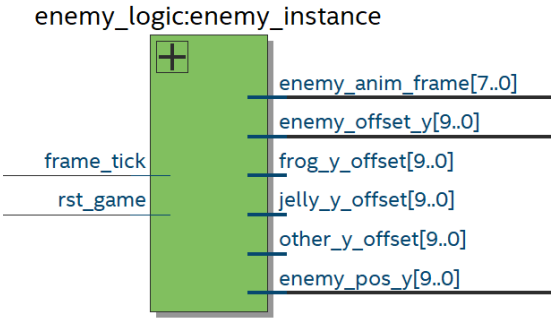
Input: frame_tick, [29:0] enemy_flags, [2:0] life_count, [7:0] kbd_code, rst_game, [9:0] enemy_offset_y, [5:0] remaining_enemies
Output: [3:0] curr_state
Description: This module implements the finite state machine (FSM) for the game, transitioning between states like "Start Screen", "Gameplay", "Win", and "Game Over" based on user input, player lives, and enemy status. It uses keyboard codes and enemy/player conditions to determine the current game mode.
Purpose: Its purpose is to coordinate overall game progression and control what phase of the game is currently active.

Module: player_control



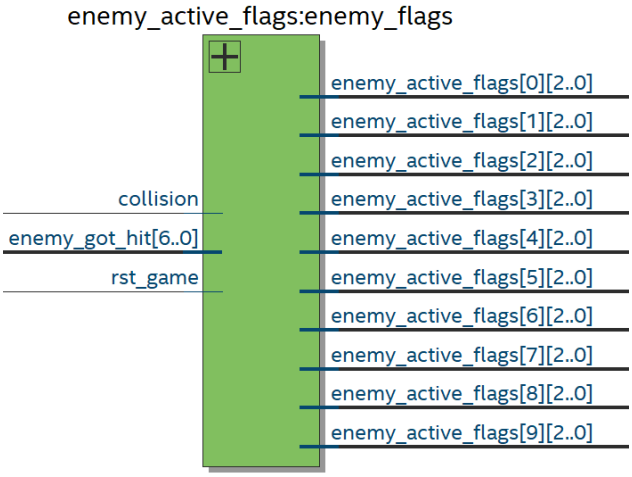
Input: rst_game, frame_tick, [7:0] kbd_input, player_collision, collected
Output: player_fire, flash_effect, [9:0] player_x, [2:0] player_life_count, invincible_flag
Description: This module handles player movement, firing, health tracking, and invincibility logic based on keyboard inputs, game events, and collisions. It maintains the player's position and life count, and controls temporary invincibility with a flash effect after being hit.
Purpose: Its purpose is to manage the player’s state and interactions in the game, ensuring proper responses to user input and game conditions.

Module: enemy _logic



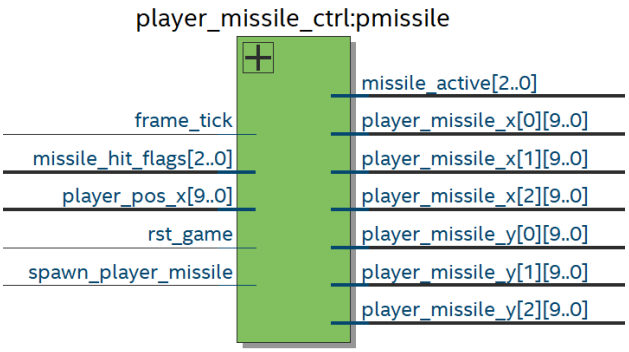
Input: frame_tick, rst_game
Output: [7:0] enemy_anim_frame, [9:0] enemy_pos_y, [9:0] frog_y_offset, [9:0] jelly_y_offset, [9:0] other_y_offset,[9:0] enemy_offset_y
Description: This module manages the horizontal and vertical movement of enemy sprites, updating position based on internal timing counters. It creates a zig-zag motion with gradual downward movement over time.
Purpose: Its purpose is to animate enemy movement across the screen in a smooth, controlled way consistent with Space Invaders-style behavior.

Module: enemy_active_flags



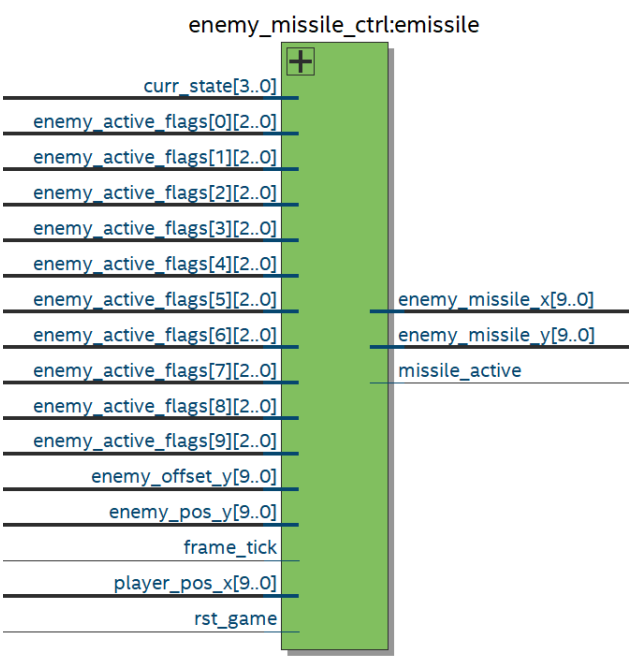
Input: rst_game, [6:0] enemy_got_hit, collision
Output: [9:0][2:0] enemy_active_flags
Description: This module tracks the active status of each enemy in a 10-column by 3-row enemy grid, marking them as "alive" or "eliminated" based on collision events. It resets all enemies to active on game restart and deactivates individual enemies when hit.
Purpose: Its purpose is to maintain a live enemy map that other game modules can reference for collision, scoring, and missile targeting.

Module: player_missile_ctrl



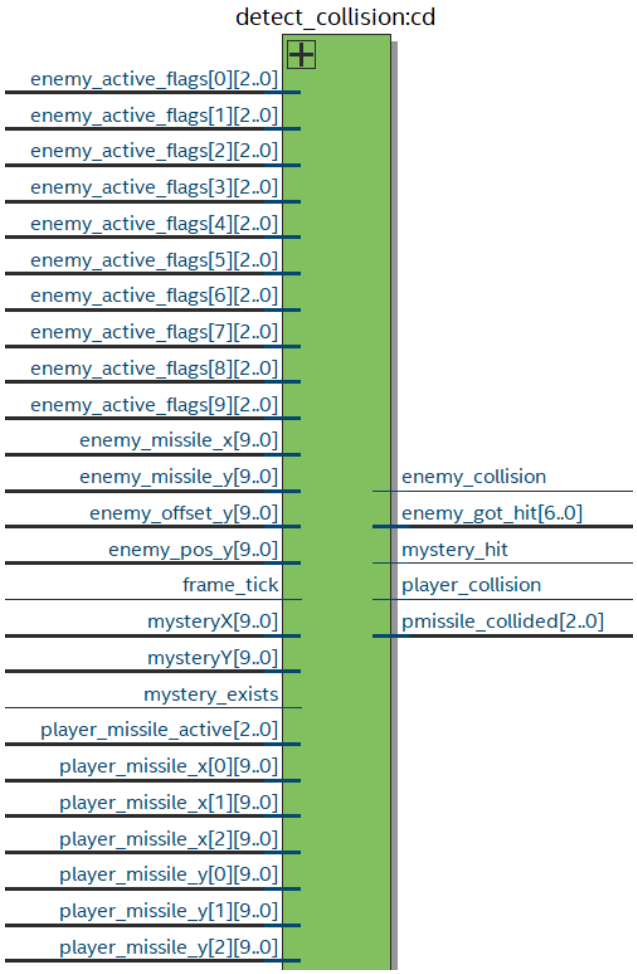
Input: rst_game, [9:0] player_pos_x, frame_tick, spawn_player_missile, [PMAX-1:0] missile_hit_flags
Output: [PMAX-1:0] missile_active, [9:0] player_missile_x [PMAX], [9:0] player_missile_y [PMAX]
Description: This module manages the lifecycle of up to PMAX player-fired missiles, including spawning missiles, updating their positions, and deactivating them on collision or when they leave the screen. It listens for a firing command and allocates missiles to the next available slot.
Purpose: Its purpose is to control player missile behavior and ensure consistent missile firing and removal throughout gameplay.

Module: enemy_missile_ctrl



Input: rst_game, [9:0] player_pos_x, [9:0][2:0] enemy_active_flags, frame_tick, [9:0] enemy_pos_y, [3:0] curr_state, [9:0] enemy_offset_y
Output: missile_active, [9:0] enemy_missile_x, [9:0] enemy_missile_y
Description: This module controls the timing, spawning, and vertical movement of a single enemy-fired missile, choosing the launch column based on the player's position and selecting the lowest active enemy in that column as the missile's origin. It updates the missile's position each frame and handles deactivation after hitting the player or reaching the bottom of the screen.
Purpose: Its purpose is to simulate enemy missile attacks in a dynamic and targeted way.

Module: detect_collision



Input: rst_game, frame_tic, [PMAX-1:0] player_missile_active, [9:0] player_missile_x [PMAX], [9:0] player_missile_y [PMAX], [9:0] enemy_missile_x, [9:0] enemy_missile_y, [9:0] player_pos_x, [9:0] enemy_pos_y, [9:0] enemy_offset_y, [9:0][2:0] enemy_active_flags, mystery_exists, [9:0] mysteryX, [9:0] mystery

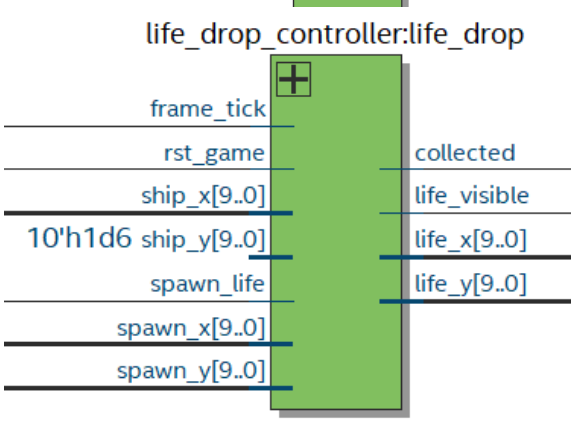
Output: [PMAX-1:0]pmissile_collided, [6:0] enemy_got_hit, player_collision, enemy_collision, mystery_hit

Description: This module checks for collisions between player missiles, enemy sprites, the mystery ship, and the player itself by comparing positions in real-time. It flags hits, identifies which enemy was struck, manages mystery ship detection, and signals player damage.

Purpose: Its purpose is to handle all hit detection logic in the game and provide real-time feedback to other modules for scoring, animation, and game state changes.

Module: life_drop_controller

Input: frame_tick, rst_game, spawn_life, [9:0] spawn_x, [9:0] spawn_y, [9:0] ship_x, [9:0] ship_y

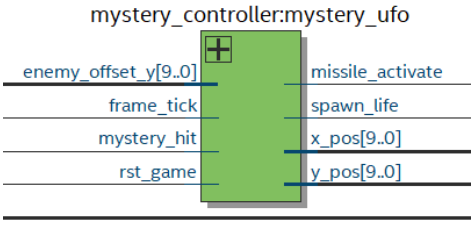


Output: [9:0] life_x, [9:0] life_y, life_visible, collected

Description: This module controls the behavior of the life drop item that appears and falls from the top of the screen when triggered. It manages its position, visibility, and collection status, detecting collisions with the player's ship and removing the drop if it's collected or falls off screen.

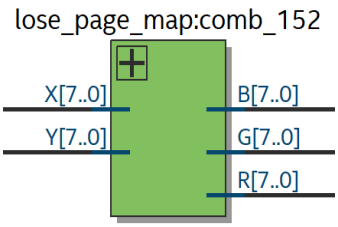
Purpose: Its purpose is to animate life drops and signal when the player successfully collects one to increase their lives.

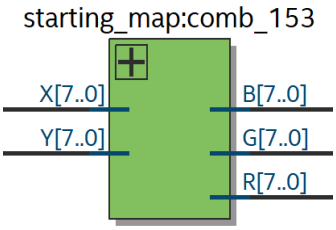
Module: mystery_controller

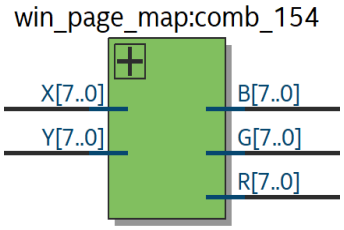


Input: frame_tick, rst_game, [9:0] enemy_offset_y, mystery_hit
Output: missile_activate, spawn_life, [9:0] x_pos, [9:0] y_pos, [9:0] mystery_y_out, [9:0] life_x_pos, [9:0] life_y_pos
Description: This module manages the mystery ship's horizontal movement, spawn timing, and random direction generation using an LFSR. It also signals when to spawn a life drop upon the ship being hit, and updates the coordinates used for placing the drop.
Purpose: Its purpose is to create a moving mystery target that occasionally triggers a life bonus for the player, adding a bonus mechanic to gameplay.

Module: starting_map & starting_rom; win_page_map & win_page_rom; lose_page_map & lose_page_rom

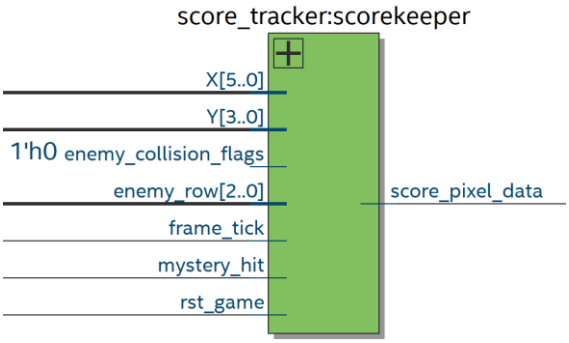






Input: [7:0] X, [7:0] Y, [13:0] address
Output: [7:0] R, [7:0] G, [7:0] B
Description: The three map modules determine which pixel color to display on the VGA screen during the “Game Over” screen by converting X and Y screen coordinates into a ROM address. The three rom modules stores 24-bit RGB values loaded from a file and outputs the color data based on the input address.
Purpose: Together, the three pairs of the module’s purpose are to display a static “Starting”, “You Win”, and “Game Over” images when the player first start the game, winning or losing the game, providing visual feedback that the game has ended.

Module: score_tracker



Input: frame_tick, rst_game, [5:0] X, [3:0] Y, enemy_collision_flags, [2:0] enemy_row, mystery_hit

Output: score_pixel_data

Description: This module tracks the player's score throughout the game by updating individual digit counters (hundreds, tens, ones) based on enemy collisions or mystery hits. It synchronizes asynchronous collision inputs, calculates score deltas, manages digit carries, and uses a digit ROM to fetch pixel data for VGA display.

Purpose: Its purpose is to maintain and display the player's current score in real time as part of the game UI.

Module: digit_rom

```
module digit_rom
(
   input [7:0] address,
   output [7:0] data
);

   parameter[0:159][7:0] ROM = {

         8'b00000000, // 0
         8'b00000000, // 1
         8'b01111100, // 2  *****
         8'b11000110, // 3 **   **
         8'b11000110, // 4 **   **
         8'b11001110, // 5 **  ***
         8'b11011110, // 6 ** ****
         8'b11110110, // 7 **** **
         8'b11100110, // 8 ***  **
         8'b11000110, // 9 **   **
         8'b11000110, // a **   **
         8'b01111100, // b  *****
         8'b00000000, // c
         8'b00000000, // d
         8'b00000000, // e
         8'b00000000, // f
          // code x31
         8'b00000000, // 0
         8'b00000000, // 1
         8'b00011000, // 2
         8'b00111000, // 3
         8'b01111000, // 4    **
         8'b00011000, // 5   ***
         8'b00011000, // 6  ****
         8'b00011000, // 7    **
         8'b00011000, // 8    **
         8'b00011000, // 9    **
         8'b00011000, // a    **
         8'b01111110, // b    **
         8'b00000000, // c    **
         8'b00000000, // d  ******
         8'b00000000, // e
         8'b00000000, // f
          // code x32
```

Input: [7:0] address

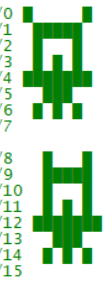
Output: [7:0] data

Description: This module is a read-only memory block that stores the bitmap patterns for digits used in the on-screen score display. Given an address that encodes both the digit value and the row index, it outputs an 8-bit slice representing one row of pixels in that digit.

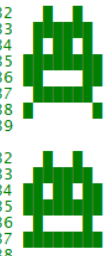
Purpose: Its purpose is to serve as a font resource that allows numerical values like scores to be displayed graphically on the screen.

Module: enemy_rom0; enemy_rom1; enemy_rom2; player_rom; mystery_rom

```
module enemy_rom0
(
   input    [7:0] addr,
   output   [15:0] data
);

   logic[7:0] addr_reg;

   parameter [0:15][15:0] ROM = {

      16'b01_00_00_00_00_00_01_00, //0
      16'b00_01_01_01_01_01_00_00, //1
      16'b00_01_00_00_00_01_00_00, //2
      16'b00_01_00_11_00_01_00_00, //3
      16'b10_10_10_10_10_10_10_00, //4
      16'b00_00_01_01_01_00_00_00, //5
      16'b00_01_00_01_00_01_00_00, //6
      16'b00_00_00_00_00_00_00_00, //7

      16'b00_00_01_00_00_00_01_00, //8
      16'b00_00_01_01_01_01_01_00, //9
      16'b00_00_01_00_00_00_01_00, //10
      16'b00_00_01_00_11_00_01_00, //11
      16'b00_10_10_10_10_10_10_10, //12
      16'b00_00_00_01_01_01_00_00, //13
      16'b00_00_01_00_01_00_01_00, //14
      16'b00_00_00_00_00_00_00_00, //15
```

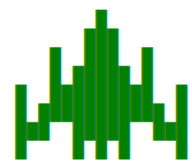


```
module enemy_rom2
(
   input    [7:0] addr,
   output   [15:0] data
);

   logic[7:0] addr_reg;

   parameter [0:15][15:0] ROM = {

      16'b00_00_01_00_00_01_00_00, //32
      16'b00_10_01_10_10_01_10_00, //33
      16'b00_10_00_10_10_00_10_00, //34
      16'b11_10_01_10_10_01_10_11, //35
      16'b11_10_00_00_00_00_10_11, //36
      16'b00_11_11_11_11_11_11_00, //37
      16'b11_00_00_00_00_00_00_11, //38
      16'b00_00_00_00_00_00_00_00, //39

      16'b00_00_01_00_00_01_00_00, //32
      16'b00_10_01_10_10_01_10_00, //33
      16'b11_10_00_10_10_00_10_11, //34
      16'b11_10_01_10_10_01_10_11, //35
      16'b00_10_00_00_00_00_10_00, //36
      16'b11_11_11_11_11_11_11_11, //37
      16'b00_00_00_00_00_00_00_00, //38
      16'b00_00_00_00_00_00_00_00, //39
```




```
module enemy_rom1
(
   input    [7:0] addr,
   output   [15:0] data
);

   logic[7:0] addr_reg;

   parameter [0:15][15:0] ROM = {

      16'b00_00_10_01_01_11_00_00, //16
      16'b10_10_11_00_01_01_11_00, //17
      16'b11_00_01_01_11_00_01_11, //18
      16'b10_01_01_01_01_01_01_11, //19
      16'b00_10_01_01_01_01_11_00, //20
      16'b10_00_00_01_00_01_00_11, //21
      16'b00_10_00_01_00_01_00_11, //22
      16'b00_00_00_00_00_00_00_00, //23

      16'b00_00_10_01_01_11_00_00, //24
      16'b00_10_01_01_00_11_01_11, //25
      16'b10_01_00_11_01_01_00_11, //26
      16'b10_01_01_01_01_01_01_11, //27
      16'b00_10_01_01_01_01_11_00, //28
      16'b10_00_01_00_01_00_00_11, //29
      16'b10_00_01_00_01_00_11_00, //30
      16'b00_00_00_00_00_00_00_00, //31
```

```
module player_rom
(
   input    [7:0] addr,
   output   [31:0] data
);

   parameter[0:7][31:0] ROM = {

      32'b00_00_00_00_00_00_00_11_00_00_00_00_00_00_00_00,  //0
      32'b00_00_00_00_00_00_11_11_11_00_00_00_00_00_00_00,  //1
      32'b00_00_00_10_00_00_11_11_11_00_00_10_00_00_00_00,  //2
      32'b00_00_00_11_00_11_11_10_11_11_00_11_00_00_00_00,  //3
      32'b10_00_00_11_01_11_10_10_10_11_01_11_00_00_10_00,  //4
      32'b11_00_11_01_11_11_10_11_10_11_11_01_11_00_11_00,  //5
      32'b11_11_11_00_00_10_11_11_11_10_00_00_11_11_11_00,  //6
      32'b11_00_00_00_00_10_00_11_00_10_00_00_00_00_01_00   //7

   };
```



Input: [7:0] addr

Output: [15:0] data; [31:0] data

Description: These modules are a read-only memory block that stores the bitmap patterns for sprites used in the on-screen enemy and the player's ship display. Given an address that encodes both the digit value and the row index, it outputs an 8-bit slice and 16-bit slice representing one row of pixels in that digit. They have 16-bit and 32-bit assigned on each since in the later, they use 2-bit of each to assign color per pixel. For example, 8'b00_01_10_11, will assign each pixel as 00 as white, 01 as red, 10 as green, and 11 as blue

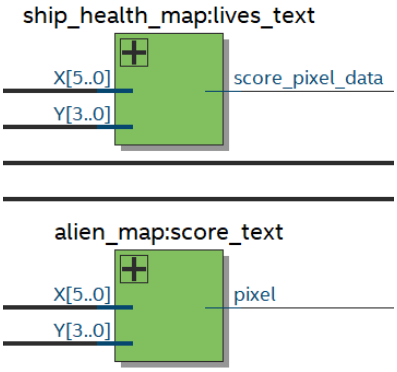
Purpose: Its purpose is to act as a compact, read-only memory block for storing and retrieving visual enemy sprite data, player sprite data, and a mystery enemy sprite data for the game.

Module: heart_rom

```
module heart_rom
(
   input    [7:0] addr,
   output   [7:0] data
);

   parameter[0:7][7:0] ROM = {

      8'b00000000,   //0
      8'b01000100,   //1
      8'b11101110,   //2
      8'b11111110,   //3
      8'b11111110,   //4
      8'b01111100,   //5
      8'b00111000,   //6
      8'b00010000    //7
```

Input: [7:0] addr
Output: [7:0] data
Description: This module holds an 8-row, 8-bit-wide sprite representing a heart icon, which is typically used to display player lives visually. It outputs a single row of the heart graphics based on the input address (vertical row index).
Purpose: Its purpose is to provide pixel data for drawing life indicators in the game, enhancing visual feedback about the player's remaining lives.

Module: alien_map & alien_rom; ship_health_map & ship_health_rom
Input: [5:0] X, [3:0] Y, [7:0] address,
Output: score_pixel_data , [15:0] data



Description: The map modules determine whether a pixel at a given screen coordinate (X, Y) should be lit as part of a static text or graphic display. It calculates a ROM address based on the X position range and row (Y), retrieves the corresponding row of pixel data from the alien_rom and ship_health_rom, and outputs a single bit to indicate whether the pixel is active. The rom modules are a read-only memory that stores bitmaps for rendering static graphics or text such as “ALIEN” and the “PLAYER’S SHIP” and symbols like colons and separators. Each address maps to a 16-bit row of pixel data, allowing flexible horizontal text or icon rendering when combined with a display module

Purpose: Its purpose is to render a small decorative or informational alien-related label on the screen. The alien one acts as a sign to show the total score the player had achieved, and the player's ship acts as a sign to show the total life left of the player.

7. Design statistics and Discussions

LUT	18147
DSP	0
Memory (BRAM)	11392 bits
Flip-Flop	2283
Frequency	128.7MHz
Static Power	106.27mW
Dynamic Power	0.77mW
Total Power	173.81mW

Table 1 Design Statistic

8. Conclusion

The following are some bugs we encountered and resolved throughout the course of our final project.

- **Discoloration on your win and game-over pictures.**
  - Issue: Using a helper tool provided through a GitHub link shared by the professor, which converts a PNG image into a .txt file containing hex codes representing pixel colors, one line per pixel. Some displayed colors were incorrect and did not match the original image.
  - Cause: The tool simplified certain hex color codes—e.g., 000000 was reduced to 000, and ce0507 was shortened to ce57. This caused the color values to be misinterpreted in my ROM module.
  - Fix: The helper tool was modified to ensure that all color codes were output as full six-digit hexadecimal values. The display colors rendered correctly after updating the text file and reloading it into the project.

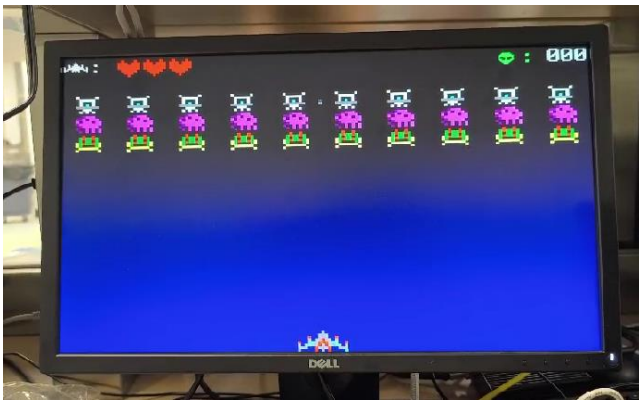
- **The player's life is not added after colliding with a life drop**
  - Issue: When the player collides with the life drop after successfully hitting the mysterious UFO, the life is not added.

- Cause: There is a mismatch of input and output from one module to another. Since this is the added module after we finish every other module, it is pretty easy to mix up the input and output and connect everything in the top_level
- Fix: We carefully track all the input and output and fix everything.

- **Keyboard Input (Spacebar) Not Working**
  - Issue: Pressing the spacebar would do nothing and would not trigger missiles nor change the game state.
  - Cause: The USB keyboard controller was incorrectly connected in the Platform Designer, and the keycode_export was incorrectly wired in top_level_space_invaders.sv.
  - Fix: Verified kbd_input values for space bar (8'h29) and properly routed the kbd_input signal from hpi_io_intf.sv module into the game_fsm.sv and player_missile_ctrl.sv. Verified with kbd_code detection logic and added edge detection if required.
- **Game State Machine Not Advancing**
  - Issue: The game was stuck in the Start state.
  - Cause: The kbd_code signal was not triggered because the conditional was incorrect or the debounce/toggle logic was missing.
  - Fix: The FSM in game_fsm.sv was modified to look for the spacebar (kbd_input == 8'h29) and move from state 0 (Start) to state 1 (Play). The win and lose conditions checked remaining_enemies == 0 and enemy_y_offset >= threshold.
- **Player Lives Not Decreasing on Collision**
  - Issue: The players were not losing lives when hit.
  - Cause: The logic in the collision detection state machine was not correctly asserting the signal to decrement life_count.
  - Fix: In detect_collision.sv, I allowed for proper detection of an overlap where the player and the enemy missile intersected. Then I modified game_fsm.sv to decrement life_count and check for it to be zero so I could trigger a loss state.
- **Missile Firing Was Unstable**
  - Issue: Players could fire many missiles very quickly or not at all.
  - Cause: The created signal was being retriggered every clock with no cooldown or edge detection.
  - Fix: I added cooldown logic using frame_tick or internal counter to prevent firing multiple times while holding the key in player_missile_ctrl.sv. I verified that create is only high for one frame for each spacebar press.
- **VGA Layer Priority Incorrect (Some Sprites Missing or Covered)**
  - Issue: Some elements (like shield blocks, mystery aliens, or heart icons) were invisible or were behind lesser-priority sprites.
  - Cause: Incorrect order in Color_Mapper.sv.

- Fix: I reorganized the priority in Color_Mapper.sv by looking at sprite-enabling signals based on the correct order, placing key UI and foreground elements after the background/gameplay pixels.

- **Mystery Alien Not Giving Score**
  - Issue: Hitting the mystery alien did not add a +10 score.
  - Cause: No flag was set upon collision, or the scoring module did not look for a mystery alien hit.
  - Fix: In mystery_controller.sv, a flag for the collision was added; the scoring logic was altered to add +10 when this flag was high—confirmed proper reset of mystery alien after it is hit.
- **Score Display Glitchy**
  - Issue: The post-collision score did not update properly.
  - Cause: Collision flags missed or updates occurring outside of frame sync.
  - Fix: Verified collision flags, verified point values for each enemy, and synced score updates to frame_tick to avoid triggering multiple times.
- **Heart Icons Not Updating with Lives**
  - Issue: The heart icons remained static even though the player's lives changed.
  - Cause: Required logic or conditions were missing from Color_Mapper.sv
  - Fix: I passed life_count into Color_mapper.sv and conditionally enabled 1-3 heart pixels using simple threshold logic. Verified that the position and priority did not overlap the gameplay sprites.
- **Collision Detection Missed Some Hits**
  - Issue: Sometimes, player or enemy missiles go straight through the targets.
  - Cause: The detection logic misused the coordinate ranges and had a timing mismatch.
  - Fix: In detect_collision.sv, missiles, and enemy/player sprite boundaries were checked for coordinate overlap. Additionally, all the detection was made to occur at the frame_tick edge for synchronization.

This final project was fun and challenging to wrap up what we learned in ECE 385. We completed all the main features we planned, and although we could not finish some of the extra features because of other classwork, we are proud of what we accomplished. One important lesson we learned was that planning helps—ensuring all the sprites and screen positions were aligned in powers of two saved us a lot of time and trouble. Working as a team, we combined everything we learned in class to build a full project from start to finish, and it was a great way to show how far we have come.

9. Documentation